

Breaking BLE MAC Address Randomization with Allowlist-Based Side Channels and its Countermeasure

YUE ZHANG, Department of Computer Science, The Ohio State University, Columbus, United States

ZHIQIANG LIN, Computer Science and Engineering Dept, Ohio State University, Columbus, United States

Bluetooth Low Energy (BLE) is ubiquitous today. To prevent a BLE device (e.g., a smartphone) from being connected by unknown devices, it uses allowlisting to allow the connectivity from only recognized devices. Unfortunately, we show that this allowlist feature actually introduces a side channel for device tracking, since a device with the allowed list behaves differently even though it has used randomized MAC addresses. Even worse, we also find that the current MAC address randomization scheme specified in Bluetooth protocol is flawed, suffering from a replay attack with which an attacker can replay a sniffed MAC address to probe whether a targeted device will respond or not based on its allowlist. We have validated our allowlist-based side channel attacks with 43 BLE peripheral devices, 11 centrals, and 4 development boards, and found none of them once configured with allowlisting is immune to the proposed attacks. We advocate the use of an interval unpredictable, central and peripheral synchronized random MAC address randomization scheme to defeat passive device tracking (introducing 1% power consumption overhead for centrals and 6.75% for peripherals, and 88.49 μ s performance overhead for centrals and 94.46 μ s for peripherals), and the use of timestamps to derive randomized MAC addresses such that attackers can no longer be able to replay them to defeat active device tracking (introducing 3.04% overhead for peripherals, and 63.58 μ s and 20.54 μ s performance overhead for centrals and peripherals). Our field testing with a long range Bluetooth sniffer shows that 16,422 of 100,101 sniffed devices are subject to our BAT attacks. We have disclosed our findings to Bluetooth SIG and many other stakeholders in October 2020. Bluetooth SIG assigned CVE-2020-35473 to track this logical-level protocol flaw. Google assigned our findings as a high severity design flaw and awarded us with a bug bounty.

CCS Concepts: • **Security and privacy** → **Mobile and wireless security**;

Additional Key Words and Phrases: Wireless security, bluetooth privacy, identity resolution key (IRK), side channel attacks, replay attacks

ACM Reference Format:

Yue Zhang and Zhiqiang Lin. 2025. Breaking BLE MAC Address Randomization with Allowlist-Based Side Channels and its Countermeasure. *ACM Trans. Priv. Sec.* 28, 4, Article 49 (September 2025), 33 pages. <https://doi.org/10.1145/3744559>

This research was partially supported by NSF award 2112471.

Authors' Contact Information: Yue Zhang, Department of Computer Science, The Ohio State University, Columbus, Ohio, United States; e-mail: zhang.12047@osu.edu; Zhiqiang Lin, Computer Science and Engineering Dept, Ohio State University, Columbus, OH, United States; e-mail: zlin@cse.ohio-state.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2471-2566/2025/09-ART49

<https://doi.org/10.1145/3744559>

1 Introduction

Being a short-range wireless communication technology, **Bluetooth Low Energy (BLE)** is ubiquitous and has been used in numerous applications today such as home entertainment, health care, sports, retail, and even recently digital contact tracing. However, BLE devices are subject to MAC address tracking since any nearby attackers can sniff the Bluetooth packets and associate them with particular devices or even users [1–5]. This is because, when using BLE for communication, a peripheral without being connected will periodically (e.g., every 20 milliseconds [6, 7]) advertise its presence to nearby centrals with an Advertising Indication (i.e., ADV_IND) packet [8] along with its MAC address; a nearby central (e.g., a smartphone) will typically respond to the ADV_IND packet with a scan request (i.e., SCAN_REQ) containing the MAC addresses of both the central and the peripheral [2], to see whether the peripheral is a known device or not. Consequently, an attacker with a sniffer can observe MAC addresses being exchanged between Bluetooth devices for MAC address tracking attacks [1–5].

Bluetooth **Special Interest Group (SIG)** is certainly aware of MAC address tracking threats, and have specified the use of MAC address randomization using such as **Resolvable Private Address (RPA)** to protect the Bluetooth privacy. In particular, RPA allows paired devices (i.e., two devices have exchanged cryptographic keys) to resolve and recognize the peer device's MAC address using its **Identity Resolution Key (IRK)** [6]. With RPA, a Bluetooth MAC address will be randomized periodically (e.g., every 15 minutes [6]), thereby hindering MAC address tracking attacks from nearby attackers.

Unfortunately, in this article we show that MAC address tracking is still possible even though it is randomized, particularly when the BLE device enables the “filter accept list” [6] defined by Bluetooth SIG (and we call it allowlist for brevity in this article), an access control feature used by a vast majority of BLE devices (e.g., Android phones, or iPhones). More specifically, when a Bluetooth device is configured with the allowlist, it behaves differently. For instance, a peripheral would ignore SCAN_REQ from unknown devices, and only respond to SCAN_RSP for its allowed device; a central may directly connect its allowed peripherals (much like a magnet) once receiving an advertisement packet without any SCAN_REQ. Therefore, an attacker can track the sniffed MAC addresses and associate them with specific ones by using a sniffer to passively collect and analyze whether the device responded or not. Although both the two communicating devices can enforce address randomization, there could exist an interval, where one device changes its address but the other one remains the same, which consequently leaves significant footprints for attackers to track the devices based on the old and not simultaneously changed address. This allows the attacker to identify both devices across the randomization time interval (e.g., more than 15 minutes).

While the above passive attacks require an attacker to keep monitoring the advertising packets, which would be quite costly for attackers, we also find attackers do not have to do so if they can replay a sniffed MAC address to actively probe whether a peripheral or a central will respond or not, by exploiting the flaw we discovered in the current RPA randomization algorithm in the Bluetooth specification. In particular, although a randomized address in RPA is generated from a random number and a pre-shared IRK between two paired devices, the current Bluetooth protocol does not specify how the random number should be chosen (other than mentioning that the random number should neither be all 0s nor all 1s at page 2,861 in [9]), and no mechanisms are placed to prevent the reuse of an existing random number. Therefore, while an attacker cannot obtain the IRK, he or she can still track the devices across the randomization time interval by simply collecting the sniffed MAC addresses and replaying them to observe whether the devices are in the allowlist or not.

To demonstrate concretely this new MAC address tracking threat, we present a novel allowlist based side channel attack dubbed **Bluetooth Address Tracking (BAT)**. Similar to other MAC address tracking attacks (e.g., [1–5]), our attacks work against devices during the advertising

stage. Particularly, we show that an attacker can either *passively sniff* (Section 4) the Bluetooth packets to identify the peripherals and the centrals, or *actively replay* (Section 5) the sniffed MAC addresses of centrals to identify their association, or *actively replay* the sniffed MAC addresses of peripherals to attract known centrals. If the attacker knows the user who is using the identified centrals or the peripherals, an attacker can even use BAT attacks to monitor the user's behaviors, and track the user's past trajectory (e.g., where the user has been to the past) or even the real-time location. We have implemented BAT using a BLE sniffer, a customized smartphone and a Bluetooth development board, and tested our *passive* and *active* BAT attacks with 54 of our own devices and also 4 development boards, and identified 39 devices (11 centrals, 24 peripherals, 4 development boards) that use the allowlist, none of which defeats our BAT attacks currently.

We have also implemented BAT using a long range BLE sniffer and a Bluetooth development board, and tested our attacks within a seven square mile region consisting of a university campus, a hospital, a residential area, and a shopping plaza. Our sniffer has identified 100,101 Bluetooth devices through packets that are publicly advertised and discovered 16,422 of them (17.9%) with allowlisting enabled. We have also tested our *active* BAT attacks with 27 of our own devices and also 4 development boards, and currently none of them defeats our proposed attacks when configured with allowlisting.

We advocate the use of an interval unpredictable, central and peripheral synchronized RPA generation scheme to counter our passive BAT attacks (Section 6.1). To defend against our active BAT attacks, we propose adding a sequence number (which could be a timestamp) when generating the RPA to ensure that each MAC address can only be used once to prevent the replay attack (Section 6.2). We have developed a prototype of our defense named **Securing Address for BLE (SABLE)** atop **Android Open Source Project (AOSP)** [10] and the SDK of Nordic by modifying the Bluetooth protocol stack. We have tested SABLE, and our experiment result shows that (1) our passive-attack defense will have about 1% power consumption overhead for centrals and 6.75% for peripherals, and 88.49 μ s performance overhead for centrals and 94.46 μ s for peripherals; and (2) our active-attack defense will not impose any power consumption for centrals but will have 3.04% power overhead for peripherals, and 63.58 μ s and 20.54 μ s performance overhead for centrals and peripherals, respectively.

2 Background

2.1 Format and Layout of BLE Packets

Being a wireless communication technology, Bluetooth relies on packets exchanged over the air for the communication, and these packets have to follow certain formats. At a high level, as illustrated in Figure 1, the layout of a typical Bluetooth packet [6] consists of: (i) the preamble for frequency synchronization, (ii) the access address for frequency identification (e.g., 0x8E89BED6 for advertising channel packets), (iii) the **Packet Data Unit (PDU)**, and (iv) the **cyclic redundancy check (CRC)**

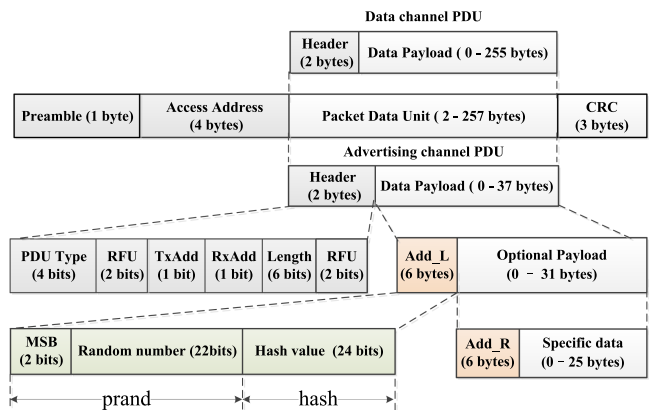


Fig. 1. Format and layout of a Bluetooth packet.

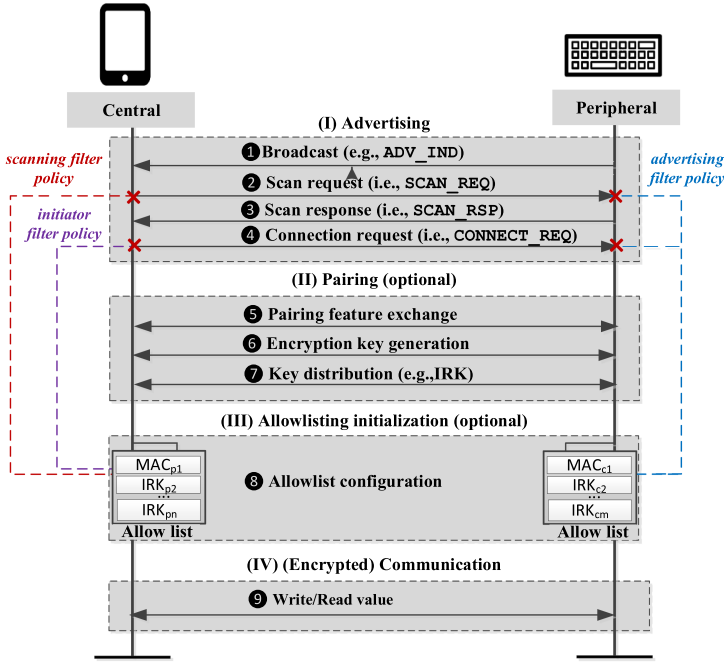


Fig. 2. BLE workflow with corresponding allowlist policies.

code for error detection. Among them, the PDU can be an advertising channel PDU for connection or a data channel PDU for communication.

An advertising PDU includes the PDU header and data payload. In particular, a packet header contains a 4-bits PDU type (e.g., `CONNECT_REQ` which indicates the device intends to connect to another device), 2-bits **Reserved for Future Usage (RFU)**, 1-bit (i.e., static or randomized) MAC address type `TxAdd` of the sender, 1-bit MAC address type of the receiver `RxAdd`, 6-bits PDU length, and the other 2-bits RFU. The data payload consists of 6-bytes local Bluetooth address `Add_L` and an optional payload from 0 to 32 bytes, which includes the data used for connection (e.g., `Add_R`, the address of the remote device which the packet sender intends to connect).

When using BLE for communications between a central (e.g., a smartphone) and a peripheral (e.g., a keyboard), it usually involves a number of steps. As illustrated in Figure 2, it could have up to nine steps, and these steps can be categorized into four stages [6]: (I) Advertising stage, (II) Pairing stage, (III) Allowlisting initialization stage, and (IV) Data exchange stage. The details of these stages are described as follows:

(I) Advertising Stage. In this stage, the central and the peripheral establish the connection by first broadcasting the presence from the peripheral, followed by a scan request from the central, then a scan response from the peripheral, and finally a connection request from the central.

— Step ❶ Broadcast. In Bluetooth communication, the presence of a peripheral must be known to the nearby centrals. This is achieved by broadcasting the packet that includes the MAC address of the peripheral, the PDU type of the advertisement (e.g., `ADV_IND` which indicates that this device can be connected and scanned, or `ADV_DIRECT_IND` which indicates this device can only be connected by devices with the expected MAC address specified in the `ADD_R` field in the broadcast packet), and other optional information such as service UUIDs and manufacture data.

Note that there is a special type of Bluetooth device, namely beacons, which only broadcast ADV_NONCONN_IND packets.

- Step ② Scan Request. When a central receives an ADV_IND packet, typically it will respond with a Scan Request (i.e., PDU type SCAN_REQ) [11]. However, since Bluetooth 4.0 (a.k.a., BLE), a new allowlist feature called *scanning filter policy* is introduced, with which a device such as a central can configure to only respond its SCAN_REQ to the allowlisted devices. This allowlist feature not only saves energy for the central, but also makes the communication more secure by only allowing connections with the listed devices. However, in practice, smartphones typically do not configure this policy as it will prevent the smartphones from discovering new peripherals.
- Step ③ Scan Response. When receiving a SCAN_REQ from a central, the peripheral typically will respond with a Scan Response [8] (i.e., PDU type SCAN_RSP). However, similar to the central, which can have a *scanning filter policy*, a peripheral can have an *advertising filter policy*, allowing the peripheral to only respond its SCAN_REQ to its allowlisted device. In addition to all the advantages of using allowlist in centrals, using allowlist in peripherals also enables security and privacy protection. In particular, Bluetooth protocol specification recommends advertising sensitive data (e.g., static data such as manufacturer information, device type such as keyboard) in SCAN_RSP (not in ADV_IND) [9] to only trusted devices (i.e., the ones in its allowlist).
- Step ④ Connection Request. When the central receives a SCAN_RSP, it then determines if the peripheral is of its interest (e.g., a keyboard that is of interest to its OS, or a blood pressure that is of interest to its corresponding app). By default, the OS of the central will not automatically initiate a Connection Request (i.e., PDU type CONNECT_REQ) to the peripherals, a user or an app has to be involved to initiate the connection. However, after the central has paired with the peripheral, it can maintain the paired peripheral into its allowlist with an *initiator filter policy* to decide whether to automatically (much like a magnet) initiate the connection whenever it sees the corresponding peripheral. The use of this allowlist policy can significantly improve the user experience, since it does not require the user to manually open the settings app of the OS or other 3rd-party apps to initiate the connection. For the peripheral, when receiving a CONNECT_REQ, it can also use the *advertising filter policy* as in Step ② to decide whether to accept this CONNECT_REQ in case that a central directly connects to it without using SCAN_REQ [9].

(II) Pairing Stage. Pairing, which is optional, is used to negotiate cryptographic keys for communication security and privacy and can be broken into three steps (from Step ⑤ to ⑦). In particular, Step ⑤ Pairing Feature Exchange: the two devices exchange their pairing features (e.g., having a display, or a keypad), which are needed to decide the appropriate pairing method such as Just Works, Passkey Entry, Out of Band, and Numeric Comparison (since Bluetooth 4.2). Step ⑥ Encryption Key Generation: the two devices determine the type of pairing method based on the features exchanged and negotiate an encryption key. Step ⑦ Key Distribution: two devices exchange keys, and these include the encryption key and also the **Identity Resolving Key (IRK)**, which is used for a BLE device to resolve its peer's randomized MAC address.

MAC address randomization is crucial for BLE security and privacy [12, 13]. A Bluetooth MAC address is a 48-bit value uniquely associated with the device. There are four types of MAC addresses: **Public Address (PA)**, **Static Random Address (SRA)**, **RPA**, and **Non-Resolvable Private Address (NRPA)**. These address types can be identified by parsing both TxAdd and the two most significant (MSB) bits of the randomized MAC address.

- **Public Address (PA)** (TxAdd=0): a PA is a globally static address assigned by the manufacturer. It never gets changed (serving as an identity for the device), obviously vulnerable to MAC address tracking attacks.

- **Static Random Address (SRA)** ($TxAdd=1$, $MSB=11$): an SRA is randomly generated by the device when it is rebooted or reset [6]. SRA is also vulnerable to MAC address tracking if the device never reboots or resets.
- **Resolvable Private Address (RPA)** ($TxAdd=1$, $MSB=01$): an RPA is generated using an IRK and it changes periodically. Only the paired device with a valid IRK can resolve the corresponding RPA to identify the known devices.
- **Non-Resolvable Private Address (NRPA)** ($TxAdd=1$, $MSB=00$): an NRPA is randomly generated and changes periodically depending on the implementation. NRPA is intended to never be resolvable by any device.

It can be noticed that only RPA can still be resolved by the peer devices if they know the corresponding IRK. This is particularly useful for a peripheral to remember the recognized centrals or vice versa. Next, we explain the format of RPA and its generation and resolution process. In particular, according to the Bluetooth protocol specification [9], an RPA consists of prand and hash. The MSB of prand for RPA is fixed (i.e., $MSB=01$), and the rest of the prand are the random bits.

- **RPA Generation.** To generate an RPA (48-bits), the central, denoted with symbol c , first selects a 24-bits prand (i.e., $prand_{24}$ whose first two bits are predefined), and then it feeds its IRK, assume irk_c , along with the selected prand into a pre-defined hash function H to get a 24-bits hash value $H_{24}(prand_{24} || irk_c)$. Finally, the RPA of c , assume rpa_c , is generated by concatenating prand and the hash value:

$$rpa_c = prand_{24} || H_{24}(prand_{24} || irk_c)$$

- **RPA Resolution.** When receiving an rpa_c from a central, the peripheral can determine whether this RPA is from its “known” device. This is achieved through the RPA resolution. At a high level, the peripheral will first split rpa_c into two parts: $prand_{24}$ and $hash_{24}$. Next, it iterates its known IRK list (each element of this list is added during the pairing), assume irk_i , to compute

$$hash'_{24} = H_{24}(prand_{24} || irk_i)$$

If $hash'_{24}$ matches the received $hash_{24}$ split from the rpa_c , then the device is resolved with the corresponding irk_i .

(III) Allowlisting Initialization (Step ③). This is also an optional stage depending on the configurations, and it is used to configure the allowlist used by early Steps (e.g., ②, ③) for device filtering. To uniquely identify a device, the allowlist feature relies on the IRKs transmitted at Step ⑦, and Step ③ just adds the IRKs to the list with other information, such as the address type. However, if the added device does not enable the address randomization, then the MAC address of the device instead of its IRK will be added to the allowlist.

Recall that there are three allowlist-based filtering policies: (i) *scanning filter policy*, (ii) *advertising filter policy*, and (iii) *initiator filter policy*. Among them, the *advertising filter policy* is deployed at the peripheral side, and the other two are deployed at the central side. Although the Bluetooth protocol does not specify how many devices can be added to an allowlist, we find that different policies may affect the number of devices that can be added to the allowlist. In the following, we summarize their practical impacts when used:

- **Scanning filter policy.** While the *scanning filter policy* is defined in the specification, it is not appropriate to be deployed on smartphones, since it will prevent smartphones from discovering new BLE devices. Instead, it is usually deployed in some customized controllers (e.g., wireless controller for video games), where one controller only discovers and communicates with only one BLE device.

- **Initiator filter policy.** This policy is widely deployed in smartphones for auto-connection without user involvement if a known peripheral is detected within its reach. Since it will not prevent smartphones from discovering and connecting new BLE devices, a smartphone with this policy enabled will be able to automatically connect multiple peripherals for user convenience.
- **Advertising filter policy.** We find that in practice only one allowlisted central can be added when enabled the *advertising filter policy* in a peripheral. This is because once the peripheral has paired with a particular central, it usually restricts other centrals to connect and pair with it. Without going through the pairing process, other centrals will not be able to deliver their IRKs to it and the allowlist will not be able to add any new centrals. As such, this policy usually allows a consistent one-to-one mapping between the allowlisting peripheral and the listed central.

(IV) **Data Exchange Stage (Step ⑨).** After the first three stages, the two devices can now exchange data using a client-server (C/S) mode, either using encryption if they have exchanged cryptographic keys or plaintext if not. Specifically, the central plays the client role, and the peripheral acts as a server providing services to the client. A read request can be sent to the peripheral if the central needs to read data from the server, or with a write request if the central needs to submit data to the server.

3 Overview of BAT Attacks

3.1 Key Observation

A key observation of our BAT attacks is that once a device has been configured with allowlisting, it will behave differently when responding to its allowlisted devices and non-allowlisted devices, as described in Section 2.1. For example, if a peripheral enables *advertising filter policy* to filter the SCAN_REQ from the nearby centrals, it will always respond to its allowlisted devices while ignoring the SCAN_REQ sent from the non-allowlisted devices. Specifically, as illustrated in Figure 3, at time t_1 , assume there is a peripheral p with static address SA (for simplicity, assume p uses a static address at this moment), which is broadcasting, and there are multiple centrals nearby (assume an allowlisted central c and an unknown central u). We can observe that p only accepts the SCAN_REQ from c with $MAC_{t_1}^c$ with a SCAN_RSP, but ignores the SCAN_REQ from the central u with address $MAC_{t_1}^u$. As such, it is quite straightforward to identify and track the allowlisted central c , by simply associating the randomized MAC addresses together from the sniffed packets. This is because the *advertising filter policy* creates a consistent one-to-one mapping between the allowlisting peripheral p and the allowlisted central c , enabling the attacker to uniquely associate the central of interest. For instance, as illustrated in Table 1 for the first five packets, we can notice that the peripheral with address $58:d7:8e:c7:8e:31$ only accepts the SCAN_REQ from $6d:d8:5e:c9:8a:41$ with a SCAN_RSP, but ignores the SCAN_REQ from the central with address $ad:d8:3e:a9:ba:52$. Therefore, for this peripheral, we can conclude it is a whitelisting peripheral.

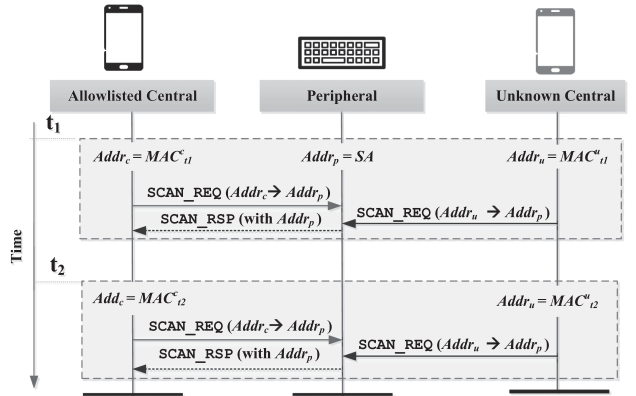


Fig. 3. An example to demonstrate the key observation.

Table 1. An Excerpt of Real World Bluetooth Packet Traces

No.	Time	Source	Destination	PDU Type
1	00:00:36	58:d7:8e:c7:8e:31	—	ADV_IND
2	00:00:40	ad:d8:3e:a9:ba:52	58:d7:8e:c7:8e:31	SCAN_REQ
3	00:00:44	58:d7:8e:c7:8e:31	—	ADV_IND
4	00:00:48	6d:d8:5e:c9:8a:41	58:d7:8e:c7:8e:31	SCAN_REQ
5	00:00:52	58:d7:8e:c7:8e:31	—	SCAN_RSP
6	00:00:56	78:e7:3e:d7:4f:2b	—	ADV_IND
7	00:01:00	7e:d7:15:94:f7:11	78:e7:3e:d7:4f:2b	SCAN_REQ
8	00:01:04	6d:d8:5e:c9:8a:41	78:e7:3e:d7:4f:2b	SCAN_REQ
9	00:01:08	78:e7:3e:d7:4f:2b	—	ADV_IND

3.2 Objectives, Assumptions, and Attack Model

Attack Objectives. We consider the users whose Bluetooth devices use RPA as our victims, since a Bluetooth device can be easily tracked if its MAC address is not randomized. Today, many Bluetooth devices have adopted randomized MAC addresses such as RPAs. For instance, Google has enforced the use of RPA on all Android smartphones since 2016 [14]. The goal of BAT attack is to show that the Bluetooth devices with RPA can still be tracked, allowing their users to be potentially deanonymized (e.g., when the MAC address can be associated to a particular user).

Without loss of generality, we define the objective of BAT attacks as follows: for a set of sniffed MAC addresses (regardless of how many BLE devices they belong to), assume that

$$G(MAC) = \{MAC_{t_1}^{dev_1}, MAC_{t_2}^{dev_2}, \dots, MAC_{t_n}^{dev_n}\}$$

where $MAC_{t_x}^{dev_x}$ is the MAC address of device dev_x at the time t_x . For any two MAC addresses, assume that

$$MAC_{t_a}^{dev_a} \in G(MAC), MAC_{t_b}^{dev_b} \in G(MAC), |t_a - t_b| > T$$

where T is the randomization time interval (e.g., 15 minutes), the goal of BAT attack is to determine whether $dev_a = dev_b$. If so, the attacker successfully associates the two MAC addresses. For example, assume at time t_a the attacker observed a victim is at her office and sniffed an address $MAC_{t_a}^{dev_a}$, and at time t_b , the attacker sniffed an anonymous device with $MAC_{t_b}^{dev_b}$ in a Starbucks. If the attacker identifies that $MAC_{t_a}^{dev_a}$ and $MAC_{t_b}^{dev_b}$ are the ephemeral addresses of the same device (e.g., a user's smartphone), the attacker then successfully associates these two addresses, and knows the user is in or near the Starbucks.

Victim Settings. Not all users whose BLE device is configured with RPA can be attacked, and instead our BAT attacks have the following two requirements:

- We require the victim to have a paired central and peripheral, and either the central or the peripheral has configured the allowlist. This is true for most centrals and peripherals today, since centrals often configure the *initiator filter* for automatic connection with known peripherals, and many peripherals often configure the *advertising filter* to respond only to known centrals.
- We require that the central and the peripheral are not always connected, which is also often true. For example, a smart speaker is disconnected from the mobile when the user is away, and earbuds are disconnected from the mobile when they are in their charging cases. If they are connected, attackers have to wait until either the two devices begin to communicate, or one of them is available for attackers to communicate. Although the attacker can use jamming to

intentionally disconnect devices, users may notice that two connected devices are disconnected, which will break the stealthiness of the attacks.

Attack Scenarios. Since the attacker is able to either (i) passively sniff the BLE traffic or (ii) actively modify the traffic, it leads to two types of BAT attacks:

- **Passive BAT Attacks (Section 4)**, which just passively sniff the advertising packets between centrals and peripherals, to see whether a peripheral will selectively respond to SCAN_REQ (i.e., reply with a SCAN_RSP or not); if so, attackers use this single bit to associate (i.e., track) the corresponding centrals into unique ones based on the observed MAC addresses.
- **Active BAT Attacks (Section 5)**, which actively manipulate the packets, e.g., forging new packets, or replaying the old packets, to observe how a peripheral or a central behaves. If there are distinctive behaviors observed, their corresponding MAC addresses are associated too.

Attacker’s Assumptions. When launching BAT attacks, we assume the attackers to have the following capabilities:

- When launching passive BAT attacks, we assume that attackers can keep collecting the advertising packets (e.g., ADV_IND, SCAN_REQ, SCAN_RSP) sent from the victim’s devices, across the randomization time interval. To this end, the attacker has to be close to the victims to sniff the Bluetooth packets, and the victim devices have to be turned on (e.g., many battery-powered peripherals may enter the sleep mode when they are not in use for a while and the attacker will have to wait until they have been awakened up).
- When launching active BAT attacks, in addition to assuming that attackers can collect Bluetooth packets, we also assume that they can actively replay and forge the advertising packets (which are not encrypted). This can be easily achieved using Bluetooth development boards or smartphones.

While we have made these assumptions, we believe that our BAT attacks are still practical compared to other Bluetooth device tracking attacks. Particularly, the first assumption is an assumption for all the Bluetooth tracking attacks (e.g., [3, 15–17]). This is because Bluetooth is designed for short range communication, and the Bluetooth packets can always be collected by either using Bluetooth sniffers at multiple locations or moving the sniffer around to track potential victims. Moreover, we can also get rid of this assumption by considering smartphones to be sniffers once controlled by malware. In particular, a malware can be installed onto smartphone to force a mobile device to work as a sniffer (e.g., as in BadBluetooth [18] and Bluetooth Misbonding [19], which assumed that the malware can be installed onto mobiles), our attacks then will not have any range limitations. However, for simplicity and practicality, we do not assume that we have a malware-controlled smartphone sniffer, though nation-state actors can have such a capability with 0-click exploits [20]).

3.3 Scope

Victim Scope. While our BAT attacks can be launched to work against all the devices that satisfy the victim’s settings (discussed in Section 3.2), we particularly focus on the following types of victims. (i) Both the central and peripheral have enforced the randomized MAC addresses. Although in Section 3.1 we use the case where the central uses random MAC addresses but the peripheral uses a static MAC address to explain the key observation of our attacks, we do not consider such a vulnerable case since the Bluetooth specification has already offered the defenses for these attacks – both devices enforce the MAC address randomization. (ii) Multiple BLE devices are nearby. This is because if there is only one device (i.e., the victim) at a specific location, and if the attacker knows such a fact, he or she can easily identify the victim device without launching our attacks

at all. (iii) While our attacks can track both the peripherals and the centrals, we focus on tracking victim centrals for proof-of-concept since many peripherals (e.g., keyboards and mice) are stationary.

Attack Scope. We also would like to narrow down the scope of our attacks. (i) We only study the BAT attacks in the advertising stage (where devices may exchange ADV_IND, SCAN_REQ, SCAN_RSP and CONNECT_REQ), and we do not consider the tracking attacks in the other three stages, as the allowlist only takes effects at this stage. For instance, we do not consider attacks such as BIAS [21], KNOB [22], and Downgrade [23] (those attacks are usually launched in the communication stage or the pairing stage), although they can potentially be used for device tracking. (ii) While our attacks can be targeted (e.g., the attacker knows a MAC address of a specific device) and untargeted (e.g., the attacker does not know such a MAC address), we use targeted attacks to demonstrate the impacts. Specifically, for proof-of-concept, we assume that for a particular targeted user (e.g., Alice), we know an instance of the MAC address of her BLE device (e.g., her smartphone). There are multiple ways to learn such MAC addresses (e.g., physically following Alice with a sniffer), and similar to other works attacking Bluetooth privacy (e.g., [24],[25]), we consider those to be out of the scope of this work. Please also note that without knowing such a MAC address, the attacker can still launch untargeted attacks to track arbitrary users if they are in the attacker's range.

4 Passive BAT Attacks

Our passive BAT attacks only sniff the broadcasting traffic of BLE devices and rely on the different response signals from devices with allowlist to associate (i.e., track) the sniffed MAC addresses. Note that we focus on the scenario in which both centrals and peripherals have used RPA-type of MAC addresses; otherwise, it is trivial to recognize the allowed centrals if the peripheral does not change its MAC address as illustrated in Section 3.1. Theoretically, when both peripheral and central have used RPA perfectly, the BAT attack, which relies on the different response signals, will fail. However, we notice that, although both the central and the peripheral will change their addresses, the address randomization usually occurs independently (without any synchronization), and there is often an interval, where one of the devices changes its address and the other does not, leaving footprints for an attacker to track Bluetooth devices across randomization time interval. To be more specific, the root causes of the vulnerability for such passive attacks are:

— **Predictable Randomization Time Interval.** The Bluetooth devices usually use predictable intervals (e.g., 15 minutes). As such, attackers can use a timing side channel to easily link a specific device by observing that the device changes its MAC address with fixed time intervals. For example, if there is one smartphone and it always changes addresses every T_r minutes, a passive attacker can observe there is one device re-randomizes its addresses at time t_1 , $t_1 + T_r$, $t_1 + 2 * T_r$, respectively. Then, it would be trivial for the attacker to link the device though it has changed its MAC.

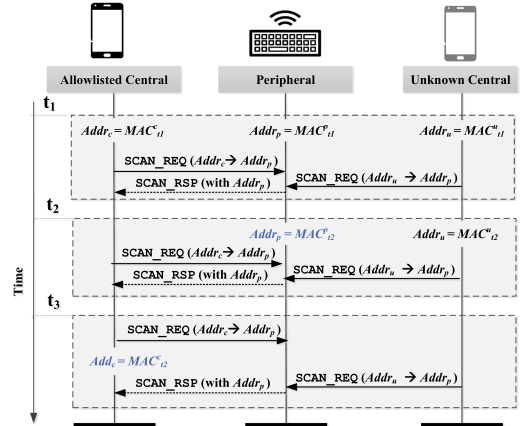


Fig. 4. Workflow of our passive BAT attacks.

—**Asynchronized Randomization.** Even if both the central and the peripheral use unpredictable randomization time intervals (e.g., both devices change MAC address randomization time interval independently), there is no guarantee that the two devices will simultaneously change their addresses. For example, assume that there are two devices, and both of them change their MAC addresses every 15 minutes. For the first time, the central changes its MAC address at t_0 , but the peripheral changes its MAC address at t_1 ($t_0 \neq t_1$). As such, there is the interval $t_1 - t_0$, where one device changes its address and the other remains the same.

We now explain the attacks in greater detail. Since broadcasting traffic starts from peripherals, we have to first identify the allowlisting peripheral from the sniffed BLE packets, and then associate the randomized sniffed MAC addresses to the corresponding centrals. To be more specific:

- Allowlisting Peripheral Identification.** To identify whether a peripheral enables allowlist or not, we can just observe how a SCAN_REQ gets responded (i.e., whether the peripheral only responds to the SCAN_REQ sent from a specific central). Although SCAN_RSP only contains the source of the response (i.e., it contains the MAC address of the peripheral but not the MAC address of the central) and we cannot know the destination of the response, we can observe which SCAN_REQ (which contains both central's MAC address and the peripheral's MAC address) triggers the SCAN_RSP. As shown in Figure 4, if we observe (i) a peripheral always ignores the SCAN_REQ containing MAC addresses of other centrals (e.g., $MAC_{t_1}^u$), the peripheral is an allowlisting peripheral; (ii) the allowlisting peripheral receives a SCAN_REQ containing the MAC address of a specific central (e.g., $MAC_{t_1}^c$), the peripheral sends a SCAN_RSP, we know that the MAC address belongs to the allowlisted central.
- Address Association.** Since the central and the peripheral will not change their addresses at the same time, the attacker can always observe an interval, during which one device does not change its address, and such a pattern can be used to track its peer. For example, in Figure 4, assume that the peripheral first changes its address to $MAC_{t_2}^p$, while the central MAC address remains $MAC_{t_1}^c$ during the small interval $T = t_3 - t_2$. Then, by observing that the peripheral with $MAC_{t_2}^p$ only responds to the central with $MAC_{t_1}^c$ and ignores the SCAN_REQ from others, we know that the peripheral is likely to be the same peripheral (or another peripheral whose allowlisted central is also the same central). Later, at t_3 , when the central changes its address, the peripheral remains $MAC_{t_2}^p$. Again, by observing the central to which the peripheral with $MAC_{t_2}^p$ always responds, we conclude that the central is the same central (given the peripherals can only add one allowed central, as discussed in Section 2.1). Note that since the attack workflow for the case where the central first changes its MAC address is similar to the case described above, we omit its details for brevity.

Attack Example I: Monitoring a Victim's Behavior. The passive BAT attack is particularly useful to monitor the user's behaviors in a specific location (e.g., user's house), which may breach user's security and privacy. Assuming victim Alice is using a stationary allowlist-enabled Bluetooth-keyboard in her home to connect her smartphone, a passive attacker is able to sniff the exchanged packets (up to 2,000-ft away when using an amplified antenna such as RP-SMA-R/A [26]) between the keyboard and the smartphone, so that the attacker is able to know the latest address of the Alice's smartphone via passive attacks. Then, whenever the smartphone communicates with any peripherals, the attacker is able to associate the peripherals to Alice and infer Alice's privacy-sensitive behavior. For example, the attacker could infer that Alice may be plagued by hyperglycemia or other blood glucose disorders if the attacker observes that her smartphone communicates with a Bluetooth blood glucose monitor (the blood glucose monitor can be recognized from the services UUID [17] provided by the Bluetooth device).

As discussed, the passive attack requires the attacker to keep monitoring the traffic between the two devices, and it will fail when the central and the peripheral are far away. Therefore, we have to look for other techniques, and this leads to the design of our active BAT attack, with which attackers can actively inject (e.g., via forging) traffic to BLE devices to observe how they will respond, and this practice does not require the attacker to keep monitoring the traffic. This is possible, since it is extremely easy for an attacker to program a malicious central (e.g., a smartphone) or a malicious peripheral (e.g., a development board) to broadcast arbitrary packets of interests. Therefore, the key question becomes what kind of packets the attacker has to forge.

When inspecting the BLE traffic of the advertising stage, as shown in Figure 2, we can notice that there is no encryption, and thus the attacker can inject arbitrary packets. The only secret is the randomized MAC addresses sent by centrals and peripherals. However, when devices using allowlist, they must use RPA that relies on the exchanged IRKs for their randomized MAC addresses generation. Therefore, if we can replay the sniffed RPA-type MAC addresses to probe how a central or a peripheral would respond, then the attacker can still launch the BAT attack.

Surprisingly, we find that the current RPA generation algorithm unfortunately never considers replay attacks, and our active BAT attack can indeed work. In particular, based on the RPA generation and resolution process described in Section 2, we find that the current Bluetooth specification only requires that a random number be used in the RPA generation, but it does not specify how to choose this random number (e.g., whether the device can choose a previously used random number). As a result, a randomized MAC address generated from a previously used random number is also considered valid, even though the attacker does not know the corresponding IRK. Therefore, a malicious central or peripheral can create a spoofed packet with a previously used MAC address to probe the peripheral or central. Since the replay attack can be launched from either a malicious central (Section 5.1) or a malicious peripheral (Section 5.2), we describe these two types of active BAT attacks, respectively.

5 Active BAT Attacks

5.1 Active BAT from Malicious Centrals

The goal of a malicious central based active BAT attack is to use a malicious central to first identify the allowlisting peripheral, from which to further track the allowlisted central. As illustrated in Figure 5, in addition to using the same approach to identify the allowlisting peripheral as in the passive BAT attacks (described in Section 4), it only adds one additional step of actively injecting a SCAN_REQ at $t_2 + \Delta$ with the harvested $MAC_{t_1}^c$ at t_1 , where $|t_2 - t_1| > T$, to probe whether the MAC address randomized peripheral responds to this injected address. If so, the peripheral is identified first, from which attackers can then track the potential centrals by actively injecting the collected MAC addresses of these centrals.

More specifically, as illustrated in Figure 5, at $t_2 + \Delta$, an attacker can use a malicious

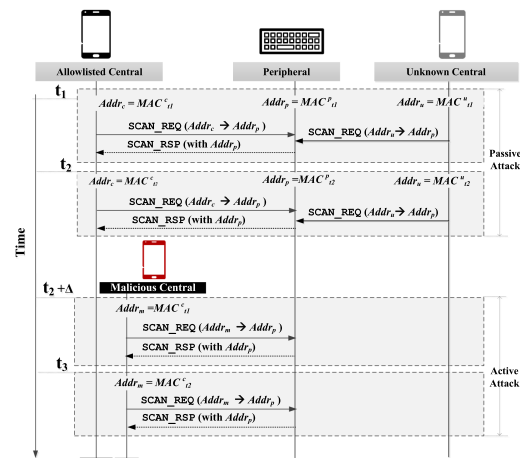


Fig. 5. Workflow of BAT attacks from malicious centrals.

central to create a SCAN_REQ with the previously used $MAC_{t_1}^c$ (which was sniffed somewhere), and then sends this request to *all* allowlisting peripherals. If one of them (e.g., the peripheral with a $MAC_{t_2}^p$) responds with a SCAN_RSP, the attacker identifies that this peripheral p is the one paired with the victim central (having the central's IRK and recognizing it). At time t_3 , the attacker can further associate the address of $MAC_{t_2}^c$ to $MAC_{t_1}^c$, since the attacker knows that the identified p is supposed to only respond to this allowlisted central c . The active attacks from malicious centrals are more powerful than passive attacks, and they can be used to track both the (past) trajectory and real-time locations of a victim. In the following, we provide two examples to demonstrate how these can be achieved.

Attack Example II: Tracking a Victim's Past Trajectory. Assume that Alice is using her smartphone to communicate with her stationary allowlist-configured Bluetooth keyboard p in her workspace, and the attacker is able to collect one of its MAC addresses (assume $MAC_{t_1}^p$). When at t_2 Alice is away from her workspace, the attacker aims to know where Alice has been to. To this end, the attacker deploys beacon-like sniffers (everywhere or just a few targeted places) that broadcast the ADV_IND packets with $MAC_{t_1}^p$ to collect the MAC addresses of all nearby centrals in their SCAN_REQs (assume $MAC_{t_2}^{c_i}$, $MAC_{t_2}^{c_j}$, $MAC_{t_2}^{c_k}$). Then, at t_3 , the attacker moves closer to Alice's workspace, and uses a malicious central to replay the collected SCAN_REQs with $MAC_{t_2}^{c_i}$ to Alice's p to test whether it responds. If so, the attacker knows c_i is Alice's phone, and if $MAC_{t_2}^{c_i}$ was collected from Starbucks (based on the sniffer's location), the attacker knows Alice was in (or near) the Starbucks.

Attack Example III: Tracking a Victim's Real-time Location w/ Tunneling. In our attack example II, the attacker has to wait at t_3 to detect Alice's past trajectory, because there is no direct communication channel between the wild centrals and p . Therefore, if the attacker is able to build a tunnel to relay the sniffed SCAN_REQs directly to p in Alice's workspace, then he or she would be able to know Alice's location instantly. This leads to our 3rd attack example, which is to additionally build a tunnel between the wild sniffers and p using attacks such as the wormhole attack [27]. Details are omitted since such a tunneling attack is well-known, and also the rest is similar to attack example II.

5.2 Active BAT from Malicious Peripherals

Using malicious centrals to probe true peripherals relies on the allowlist of the *advertising filter policy* in the peripherals. However, not all peripherals enable this policy, and rather many centrals (e.g., Android mobiles, iPhone, and Windows tablets) have enabled the *initiator filter policy*, which will instantly respond to the "known" peripherals (by storing the peripheral's IRK) once they are in their range. Therefore, we design another active attack by using spoofed packets generated from a malicious peripheral, which broadcasts the advertising packets to all nearby centrals, and only the central with the *initiator filter policy* enabled will respond, allowing an attacker to instantly know a central's location. Similar

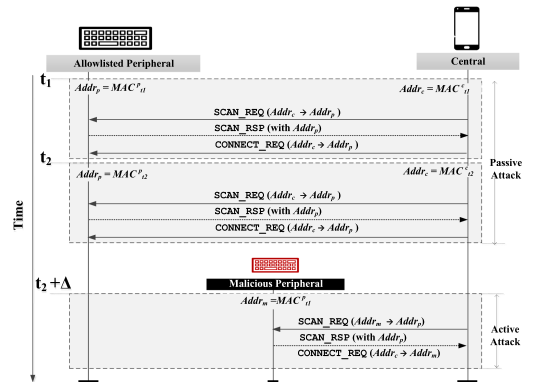


Fig. 6. Workflow of BAT attacks from malicious peripherals.

to active BAT attacks from malicious central, this attack also does not need uninterrupted observation of Bluetooth traffic.

More specifically, as illustrated in Figure 6, assume an attacker has observed that a central with $MAC_{t_1}^c$ initiated a connection with a peripheral with $MAC_{t_1}^p$ at t_1 . At t_2 , the central changed its address to $MAC_{t_2}^c$ and the peripheral changed its address to $MAC_{t_2}^p$. To associate whether the address $MAC_{t_1}^c$ and $MAC_{t_2}^c$ belong to the same central, at $t_2 + \Delta$, the attacker can simply create a malicious peripheral to broadcast its presence with the harvested $MAC_{t_1}^p$ or $MAC_{t_2}^p$. If the victim central has enabled the *initiator filter policy*, it will automatically initiate CONNECT_REQ to the malicious peripheral. Consequently, the attacker is able to associate $MAC_{t_1}^c$ with $MAC_{t_2}^c$ and identify the targeted central.

Attack Example IV: Tracking a Victim's Real-time Location w/o Tunneling. Building a tunnel might be expensive. There is no need to do so if attackers use malicious peripherals to associate centrals with the *initiator filter policy*. Still assume that Alice uses her phone configured with the *initiator filter policy* to automatically communicate with her Bluetooth keyboard p in her workspace, and at t_1 the attacker is able to observe one of its MAC addresses $MAC_{t_1}^p$. Then later at t_2 when Alice is away from her workspace, the attacker directly uses beacon-like sniffers to advertise $MAC_{t_1}^p$ to nearby centrals, and if a central instantly connects the sniffer with a CONNECT_REQ, then the attacker knows Alice's real-time location.

6 Countermeasure

In this section, we present a countermeasure named SABLE to defend against our BAT attacks. Since the attacks can be launched passively or actively, we need the following two corresponding defenses described in Section 6.1 and Section 6.2, respectively.

6.1 Defending Against Passive BAT Attacks

Overview. The passive attacks are made possible due to the central and peripheral independently randomizing their addresses. As such, we developed our defense to synchronize their randomization time, using a logic time, which can be traced back to Lamport clock [28]. At a high level, to defend against the passive attacks, we propose to (1) make MAC randomization at both sides synchronized, and (2) make the length of the interval random; otherwise, a repeatable time interval (e.g., every 15 minutes) allows attackers to associate the randomized MAC addresses across intervals.

(I) Making Randomization Synchronized. We first discuss how the two devices change their addresses and when it is the time for them to perform the address randomization. Since the central and the peripheral may not always be close to each other (e.g., the user could take her central away, and vice versa, or one of them is turned off), and if they are not close to each other, the two devices cannot communicate to decide how they could change their addresses. Therefore, we have the following two scenarios to address:

- **(a) Two devices are close to each other.** In this case, we cannot let the peripheral and the central independently start their own randomization. Instead, we notice that the allowlisting always starts from the peripheral when advertising its presence, and then the central responds. Therefore, we can take advantage of this causality for the synchronization and let the randomization start right before the peripheral starts to send the ADV_IND at the peripheral side, and the central starts right after receiving the corresponding ADV_IND, as illustrated in Figure 7. As such, attackers will only observe an always synchronized randomization across an unpredictable

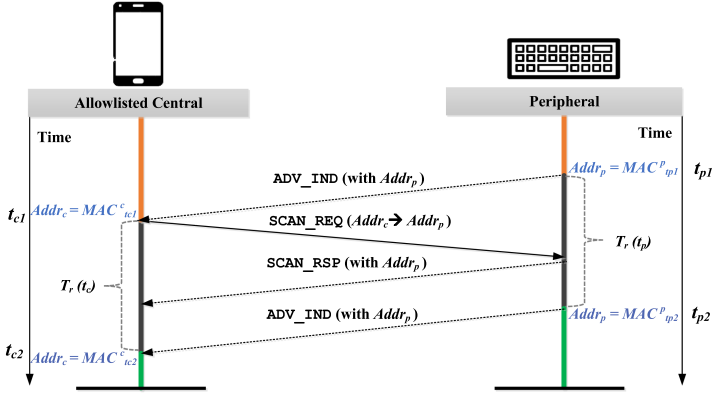


Fig. 7. Status changing in the passive defense.

interval, and they cannot associate the MAC addresses across the randomization time intervals anymore.

- **(b) Two devices are far away.** In this case, we can let the central and peripheral independently start their own randomization, since the attacker can no longer associate the addresses through observing devices' SCAN_REQ and SCAN_RSP. Assume a central and a peripheral have passed N times synchronized address randomization, and now they are in their $(N + 1)$ -th synchronized address randomization. During this interval (e.g., 15 minutes), the central is taken away by the user, and they are no longer close to each other (we use the case where the central is away from the peripheral as an example, since the other case is essentially the same). As a result, the peripheral will not be able to receive any SCAN_REQ from its allowed central. When it is the time to change its address, the peripheral will fetch its own time T_p and generate a random address rpa_p . Since the central is not nearby, the lifetime of such a generated rpa_p can be a random time period without notifying the central for synchronization. The peripheral will continue to generate new RPAs using its freshest timestamps, and each of those RPAs will have a random lifespan. When the central is back, the peripheral will resume its state by re-entering its $(N + 2)$ -th address randomization. Specifically, since the peripheral keeps broadcasting, and whenever the central is close to it, the central can recognize it by correctly resolving its RPA. Then, the central sends a SCAN_REQ, which contains its latest RPA generated from the freshest timestamp, to inform the peripheral that it is back. When the peripheral receives the RPA, it enters the $(N + 2)$ -th synchronized address randomization, and then the central also enters the $(N + 2)$ -th synchronized address randomization when it receives the ADV_IND from the peripheral. The following procedures are the same as that in scenario (a), which we will not present in further detail for brevity.

(II) Making Randomization Time Interval Consistent and Unpredictable. Our randomization time interval needs to be a secret and unpredictable to attackers. To achieve that, the two devices can introduce a new secret value or reuse existing ones (e.g., IRK) to derive the randomized intervals. The secret also needs to be updated dynamically on both sides to ensure that the randomization time intervals can also be changed simultaneously. As such, we propose to use the hash of an auto-incremented secret to derive the randomization time interval, since every time the value of such derived secret changes, the derived randomization time interval changes as well.

The devices can use an existing secret S (e.g., the LTK, or the IRK) with an automatic increment to derive the random time intervals. Assume there is an S_0 , which can be the IRK, and every time we update the interval S_j , we automatically increase it by k (which can be one or any other predefined

number) from previous S_i .

$$S_j = S_i + k$$

where S_0 is assumed to be the IRK for simplicity. Then, we can derive the length of the randomization time interval at the i th interval

$$T_r(i) = H(S_i) \bmod T_{max}$$

where T_{max} is set to be 15 minutes, and $H(S_i)$ is the hash of S_i . As such, at any given time interval i , both central and peripheral will have a pre-determined random interval $T_r(i)$ unknown to attackers once S_0 , the initial secret, is exchanged.

Also, note that there has to be a unique secret S for each paired central and peripheral. But one central (e.g., a smartphone) can be paired with multiple peripherals. Therefore, the randomization time interval and IRK must be S -specific. That is, a central needs to use a peripheral-specific RPA-type MAC address correspondingly to connect its paired peripherals. Finally, two paired devices may be out of synchronization, e.g., it is supposed to be i th interval, but for some reasons (e.g., one of them lost its battery) the peripheral or the central may still be in the previous $(i-1)$ -th interval or even more. If this occurs, the devices have to correct the errors based on the observed intervals. In particular, when a central notices that the two devices get out of synchronization by observing whether the peripheral's interval equals to its own currently used interval, the central starts the error handling process: the central first calculates a few intervals (e.g., $(i-1)$ -th interval) that are close to the currently used interval (assume it is the i th interval) based on the algorithm of how $T_r(i)$ is derived, e.g., $T_r(i-1) = H(S_{i-1}) \bmod T_{max}$, $T_r(i-2) = H(S_{i-2}) \bmod T_{max}$, and compares those calculated intervals to check if any of them equals to the peripheral's interval. If so, the central updates its i (the current interval) accordingly, and the error is corrected (the central and the peripheral now have the same i).

To make the interval random, the two devices must keep the same secret and update it simultaneously. However, it is possible that the two paired devices may not be in synchronization. For example, it is supposed to be the i th interval, but the peripheral or the central may still be in the previous $(i-1)$ -th interval, and we have to correct such errors. Fortunately, since the allowlisting always starts from the peripheral when advertising its presence, and then the central responds, the central can identify the fact that at the very first time: if the central notices that peripheral changes the MAC address to an earlier or later one than it should be, the central knows that the two devices are out of synchronization. Therefore, we let the central always correct its interval based on the peripheral's interval.

Our synchronization error correction approach is quite simple. Assume that when errors occur, the peripheral or the central is in the previous $(i-\delta)$ -th interval, where δ is usually a very small integer (e.g., one or two) in practice, although in theory the worst case δ could be i , which would require more computational resources to correct it. When the central notices that the two devices get out of synchronization by observing whether the peripheral's interval equals to its own currently used interval, the central starts the error handling process: the central first calculates a few intervals (e.g., $(i-1)$ -th interval) that are close to the currently used interval (assume it is the i th interval) based on the algorithm of how $T_r(i)$ is derived, e.g., $T_r(i-1) = H(S_{i-1}) \bmod T_{max}$, $T_r(i-2) = H(S_{i-2}) \bmod T_{max}$, and compares those calculated intervals to check if any of them equals to the peripheral's interval. If so, the central updates its i (the current interval) accordingly, and the error is corrected (the central and the peripheral now have the same i).

Note that there could be a case where the central's calculated intervals are the same (e.g., $T_r(i) = T_r(i-1)$ due to a partial hash collision) and they all equal the peripheral's currently used interval. In this case, the central can choose either i or $i-1$, as the i th interval and wait to check

whether the peripheral's upcoming interval equals its own interval at the next time of address randomization. If so, the error is corrected to the most recent i ; otherwise, the central will need to keep performing the error correction until the error gets corrected.

6.2 Defending Against Active BAT Attacks

Overview. Fundamentally, our active BAT attack works because the current RPA-type MAC address generation suffers from the replay attack. There are multiple countermeasures to defeat a replay attack, e.g., using a sequence number, or a timestamp, or storage-based (i.e., saving all the used random numbers onto the devices). However, not all of them are practical. For example, the storage-based solution may cost huge amount of storage resource, e.g., up to $365 \times 24 \times 4 \times 48 = 1,681,920$ MB yearly, assuming that an address is changed every 15 minutes (note that each address is 48 bits).

Another well-known defense against replay attack is to add random sequence numbers. Theoretically, we can add a synchronized, auto-incremented sequence number, together with the random number prand, to generate a one-time only RPA-type MAC address. However, this defense will introduce additional storage and communication overhead, as well as additional sequence number maintenance efforts. For example, the central and the peripheral must update their negotiated sequence number simultaneously to ensure they keep the same sequence number. We therefore propose to use the timestamps together with the random number prand to generate a one-time only RPA-type MAC address, which can only be used within a given time window depending on the configuration. Note that a timestamp essentially can be considered as a sequence number, and it increases automatically as time passes by.

However, we did not use the timestamp directly since this will result in the collision of the Bluetooth MAC address. Recall that the Bluetooth MAC address is 48 bits, and if we include a 24-bit timestamp in the MAC address, we only have 24 bits to ensure the randomness of the MAC addresses. This can only result in 2^{24} (16,777,216) unique MAC addresses, which is much less than the current number of Bluetooth devices (e.g., annual Bluetooth device shipments is 4.7 billion in 2021 [29]). Nevertheless, given that a specific area may not have too many devices, using the timestamps directly could still be a viable mitigation approach.

Detailed Design. Our active defense is designed by piggybacking the existing protocol without adding any extra field in the protocol but only modifying the central and peripheral to process the timestamp. In total, there are four steps involved in our new design. Note that in the following description, we assume the peripheral has enabled allowlist (since the defense workflow for the central with allowlist is similar, we will not present it for brevity). Our defense also works when both peripheral and central enable the allowlist.

- **Step ① Key Distribution.** Assume that a peripheral p and a central c have passed through the advertising stage, and now they are entering the pairing stage. When the central distributes its IRK irk_c , it also distributes the current timestamp T_c^0 of the central (i.e., $T_c^0 = \text{getCurrentTime}(c)$ where getCurrentTime returns the number of seconds since UNIX Epoc for c).
- **Step ② Allowlisting Configuration.** The peripheral p receives irk_c , T_c^0 , and gets its current time T_p^0 via $\text{getCurrentTime}(p)$ at p . Then, it saves both T_p^0 and the received irk_c , and T_c^0 for future reference when resolving the RPA. Now p finishes configuring its allowlist, with which p will only respond to SCAN_REQ or CONNECT_REQ from this paired and allowlisted c .
- **Step ③ RPA Generation.** Later, when the central c receives an advertising packet from p , it uses its irk_c , a random number prand_{24} (with its two MSB bits to be 01 to denote RPA-type MAC address), and the current time T_c to generate its current

$$rpa_c = (\text{prand}_{24} || H_{24}(\text{prand}_{24} || T_c || irk_c))$$

After that, c can send this newly generated rpa_c as usual in its SCAN_REQ or CONNECT_REQ to p . Please note that this timestamp T_c should be used in all subsequent sessions within a random interval (e.g., 15 minutes) to avoid the central changing its addresses too often.

- **Step ④ RPA Resolution.** When p receives a SCAN_REQ with an rpa_c , it resolves this rpa_c using its stored irk_c as follows. First, we follow the initial RPA resolution by splitting rpa_c into two parts: the random number $prand_{24}$ and $hash_{24}$. Next, we get T'_c from calculation:

$$T'_c = \text{getCurrentTime}(p) - (T_p^0 - T_c^0)$$

After feeding the hash function H with three inputs irk_c , $prand_{24}$ and T_c to compute the hashed value:

$$hash_{24}' = H_{24}(prand_{24} || T'_c || irk_c)$$

If $hash_{24}$ equals to $hash'_{24}$, this rpa_c is resolved. If not, there could be two possibilities. (i) The time on the central and the peripheral gets out of synchronization due to unknown reasons (e.g., clock skews). In that case, the peripheral may need to enumerate a few possible timestamps within a threshold (e.g., the threshold could be one if the peripheral attempts to try the former timestamp and the next timestamp) to correct the errors and update T_c if resolved. (ii) The address is a replayed one and the peripheral should reject it. In that case, the timestamp has significant differences when compared with the saved T'_c or the current time (e.g., the difference is beyond a threshold).

7 Implementation

7.1 BAT Implementation

The passive attack can be implemented through observing the Bluetooth packets using an Adafruit LE sniffer. The active attack requires injecting spoofed packet of SCAN_REQ from a malicious central, or spoofed advertisement packet ADV_IND or ADV_DIRECT_IND from a malicious peripheral. To implement the malicious central, we customize Android 9.0 through the AOSP [10]. Specifically, we modified the source code of function `btm_gen_resolve_paddr_cmpl()` in `btm_ble_addr.cc`. The function starts to execute when the RPA of the smartphone generated and a local variable `p_cb->private_addr.address` in the function is an array that maintains the generated RPA. By manipulating the values of that array, we can change the RPA of the smartphone to an RPA of interest, e.g., a victim central. The malicious central sends SCAN_REQ automatically when it is in the broadcasting range of the whitelisting peripheral. To implement the malicious peripheral, we utilized a smartphone since it can function as a peripheral with the necessary customization. To this end, we download and install an app named BLE Peripheral Simulator from Google Play to spoof the victim centrals.

7.2 SABLE Implementation

For the active defense, since we have modified the existing BLE protocols, we have to patch the OS of both the central and peripheral. To this end, we implemented active defense using Google Pixel 2 as a central and the Nordic NRF52 development board as a peripheral. On the smartphone, we also modified `btm_ble_addr.cc` to generate the RPA address. The current time is obtained through the function `getTimeOfDay()`. For the peripheral, we customize the source code of `id_manager.c`, which is provided by Nordic SDK for RPA generation and resolution. Particularly, the function `im_address_resolve()` is used to resolve the RPA of the central, and we modified the source code of that function to allow it to resolve our modified RPAs. The current time of the chip is obtained through `app_timer_cnt_get()`, which returns the current time on the chip in

the format of tick, which can be converted to UNIX Epoc time based on the system frequency of the chip. To implement the passive defense, on the central side, we store the secret at `ble.conf`, and modify `btm_ble_addr.cc` to control the address generation cycles. The randomization cycle on the development board can be configured through the parameter `private_addr_cycles_S`. To store the secret, we create a key-value map in long-term storage by using C structure `fds_flash_record_t`.

8 Evaluation

We have implemented both our attacks and defenses for the evaluation. To implement our passive BAT attack, we used Adafruit LE sniffer [30] to collect BLE advertising packets (i.e., `ADV_IND`, `SCAN_REQ`, `SCAN_RSP` and `CONNECT_REQ`). To implement our active BAT attack, we customized Android 9.0 through the AOSP [10] to implement both the malicious central and the malicious peripheral. To implement our defense, we used Google Pixel 2 as a central, and a Nordic NRF52 development board as a peripheral for testing our SABLE. In the rest of this section, we present our evaluation results.

8.1 Experiment Setup

We first launch our BAT attacks against our own devices configured with allowlist. Intuitively, keyboards, mice, earbuds, and smart watches usually support a single user, and they tend to have an allowlist. We therefore first purchased 43 Bluetooth peripherals such as keyboards, earbuds, and mice, and among them, 24 of them have the allowlist enabled. Other 19 peripherals, which do not have allowlist, are out of our focus. For the central devices such as smartphones, fortunately they all have the allowlist enabled, and therefore we used all of our own Bluetooth centrals (six smartphones, three laptops, and two tablets). We also purchased a Bluetooth development board each from TI, Nordic, Silicon Labs, and Cypress, respectively. As reported in Table 2, we therefore eventually have 39 tested devices in total (11 centrals, 24 peripherals, 4 development boards).

Field Test Settings. Our attack can be used to understand how many peripherals in the wild used the allowlist, and deanonymize their centrals if so. To this end, we passively captured 2,248,198 packets (those packets are from both centrals and peripherals) in a university campus and its nearby zones. While we would like to make the dataset publicly accessible, it may be abused by attackers. Therefore, we would like to provide some details regarding the statistics and how we have collected these data:

- **Sniffer Capability.** Typically, BLE signals have a limited range of up to 100 meters. However, by employing specialized receiver adapters and amplifiers, an attacker can extend their ability to intercept BLE signals to a range of up to 1,000 meters. In our research, we have successfully constructed a long-range passive BLE sniffer using an Ubertooth one sniffer [31] in conjunction with a specialized BLE Antenna RP-SMA-R/A [17], as we did in BLEScope [17]. Our sniffer covers an impressive distance of 2,000 feet, and the area tested at each location encompasses approximately 1.43 square miles.
- **Timeframe.** We conducted two separate visits to each of the selected locations, spanning two different days, with each visit lasting approximately from 1 pm to 6 pm. Our objective was to gain a comprehensive understanding of the number and variety of Bluetooth devices present during both visits. To ensure accurate data collection, we took into account the dynamic nature of Bluetooth device identification. Most Bluetooth devices tend to change their MAC addresses every 15 minutes as a privacy measure. To mitigate this, we adopted a strategic approach during our fieldwork. During each visit to a specific location, we stayed in the

Table 2. Evaluation Results of the Tested Devices

Peripherals & Development Boards						Passive Attacks		Active Attacks					
Allowlist			Device Type	MAC Addr	Power Saving			From					
Brand & Model	Enabled by P	Used by C				TC	TP	TC	TP	TC	TP	TC	TP
DRACONIC	✓	✓	Keyboard	SRA	✓	✓	✓	✓	✓	✓	✓		
JellyComb	✓	✓	Keyboard	SRA	✓	✓	✓	✓	✓	✓	✓		
iClever	✓	✓	Keyboard	SRA	✓	✓	✓	✓	✓	✓	✓		
Microsoft (V1)	✓	✓	Keyboard	SRA	✓	✓	✓	✓	✓	✓	✓		
Microsoft (V2)	✓	✓	Keyboard	SRA	✓	✓	✓	✓	✓	✓	✓		
byteblue	✓	✓	Keyboard	SRA	✓	✓	✓	✓	✓	✓	✓		
Logitech K780	✓	✓	Keyboard	SRA	✓	✓	✓	✓	✓	✓	✓		
Logitech K830	✓	✓	Keyboard	SRA	✓	✓	✓	✓	✓	✓	✓		
Logitech K380	✓	✓	Keyboard	SRA	✓	✓	✓	✓	✓	✓	✓		
SXWL	✓	✓	Keyboard	SRA	✓	✓	✓	✓	✓	✓	✓		
SXWL	✓	✓	Mouse	SRA	✓	✓	✓	✓	✓	✓	✓		
Inphic	✓	✓	Mouse	SRA	✓	✓	✓	✓	✓	✓	✓		
Vogek	✓	✓	Mouse	SRA	✓	✓	✓	✓	✓	✓	✓		
JellyComb (V1)	✓	✓	Mouse	SRA	✓	✓	✓	✓	✓	✓	✓		
JellyComb (V2)	✓	✓	Mouse	SRA	✓	✓	✓	✓	✓	✓	✓		
SEENDA	✓	✓	Mouse	SRA	✓	✓	✓	✓	✓	✓	✓		
MiBand 4C	✓	✗	Wristband	PA	✗	✓	✓	✓	✓	✗	✓		
i-Home Alexa	✗	✓	Speaker	PA	✓	✗	✓	✓	✓	✓	✓		
TEZO	✗	✓	Earbuds	PA	✓	✗	✓	✓	✓	✓	✓		
Boltune	✗	✓	Earbuds	PA	✓	✗	✓	✓	✓	✓	✓		
SoundBot	✗	✓	Earbuds	PA	✓	✗	✓	✓	✓	✓	✓		
Riitek	✗	✓	Keyboard	PA	✓	✗	✓	✗	✓	✓	✓		
Cimeteck	✗	✓	Mouse	SRA	✓	✗	✓	✗	✓	✓	✓		
Ergonomic	✗	✓	Mouse	SRA	✓	✗	✓	✗	✓	✓	✓		
TI CC2640R2F	✓	✓	Dev Board	RPA	-	✓	✓	✓	✓	✓	✓		
Nordic NRF52	✓	✓	Dev Board	RPA	-	✓	✓	✓	✓	✓	✓		
Silicon Labs 6101D	✗	✓	Dev Board	RPA	-	-	-	✗	✗	✓	✓		
Cypress CY8CIT	✗	✓	Dev Board	RPA	-	-	-	✗	✗	✓	✓		
Centrals													
Allowlist						Passive Attacks		Active Attacks					
								From		From			
Brand & Model	Enabled by C	Used by P	Type & OS	MAC Addr	Random Interval	TP	TC	TP	TC	TP	TC		
												Malicious Central	Malicious Peripheral
Google Pixel 4	✓	✓	Phone (Android 11)	RPA	5-15	✓	✓	✓	✓	✓	✓		
Google Pixel 2	✓	✓	Phone (Android 10)	RPA	5-15	✓	✓	✓	✓	✓	✓		
Samsung S10	✓	✓	Phone (Android 10)	RPA	5-15	✓	✓	✓	✓	✓	✓		
Google Pixel 4	✓	✓	Phone (Android 10)	RPA	5-15	✓	✓	✓	✓	✓	✓		
iPhone 8	✓	✓	Phone (iOS 13.2)	RPA	15	✓	✓	✓	✓	✓	✓		
iPhone 11	✓	✓	Phone (iOS 13.2)	RPA	15	✓	✓	✓	✓	✓	✓		
iPad	✓	✓	Tablet (iOS 13.2)	RPA	15	✓	✓	✓	✓	✓	✓		
Dell GD1H4KU	✓	✓	Laptop (Windows 10)	PA	+∞	✓	✓	✓	✓	✓	✓		
Dell	✓	✓	Laptop (Ubuntu 20.02)	PA	+∞	✓	✓	✓	✓	✓	✓		
Thinkpad T450s	✓	✓	Laptop (Windows 8)	PA	+∞	✓	✓	✓	✓	✓	✓		
Surface Pro	✓	✓	Tablet (Windows 10)	PA	+∞	✓	✓	✓	✓	✓	✓		

“TC” means tracking the central and “TP” means tracking the peripheral. “✓” means vulnerable and “✗” means not.

area for approximately 30 minutes. This extended duration allowed us to effectively sniff and record Bluetooth packets from various devices. By doing so, we aimed to capture a more representative and stable snapshot of the devices present at each location, considering the ever-changing nature of Bluetooth MAC addresses.

- **Collected BLE packets.** We have amassed a dataset of 2,248,198 Bluetooth packets, all of which are pristine, and devoid of any extraneous noise. This impeccable quality is attributable to the Sniffer’s adept ability to filter out non-Bluetooth packets and arrange the Bluetooth packets in the correct format. Although a few Bluetooth packets may harbor minor imperfections in their payloads, our experiment only necessitates the processing of their MAC addresses.

8.2 Evaluation of Passive BAT Attacks

To test our attacks against our own centrals and peripherals, we first need to pair them up. To this end, for each peripheral, we iterate our centrals: if any of them can expose the vulnerability for the peripheral, we stop pairing more centrals and move to the next peripheral. Also, during our pairing, if a device never involves allowlist (including by its peer device), we discard it in our evaluation. As reported in Table 2, there are 17 peripherals configured with the allowlist, as shown in the 2nd column. Unfortunately, all of them are vulnerable to passive BAT attacks to track their corresponding centrals since they used static addresses, which are subject to tracking without deploying our attacks at all as described in Section 3.1. We can also notice that these peripherals include keyboards, mice, and smart wristbands. As expected, these devices are intended for exclusive use (a single user) and tend to have the allowlist. However, other IoT devices such as smart speakers tend not to have the allowlist since they are household devices (serving more than one user).

Since all the peripherals are vulnerable to the simple passive attacks described in Section 3.1, we have not validated the passive attacks introduced in Section 4. To really evaluate these attacks, we use four development boards with RPAs, and pair the smartphone with them. Among them, only TI CC2640 and Nordic NRF52 support allowlist [32, 33], and the development boards (as well as the devices that used those development boards) are confirmed vulnerable to our passive attacks. Also, while the tested centrals are all vulnerable to BAT attacks, we notice that the randomization interval is varied from the operating systems. For example, Android-11 changes its interval to every 5 to 15 minutes, but iOS stays a constant 15 minutes. Meanwhile, Windows and Linux laptops use PAs without randomization. In addition, as reported in the 6th column of Table 2, among the tested peripherals, we notice that only the wristband will always stay awake.

Table 3. Identified Allowlisting Devices and Their Features

Item	#	%
# Address Type		
# PA	3,909	23.9
# SRA	644	3.9
# NRPA	563	3.4
# RPA	11,306	68.8
# Role		
# Peripheral	16,281	99.1
# Peripheral & Central	141	0.9
# Advertising Type		
# ADV_IND	15,625	93.7
# ADV_DIRECT_IND	797	6.3

8.3 Evaluation of Active BAT Attacks

Active BAT Attacks from Malicious Centrals. Since the active BAT attacks from malicious centrals require the victim peripherals to have allowlist (regardless of whether or not they have used static addresses) and configure *advertising filter policy*, and in our passive attacks we have already identified that there are 17 peripherals configured with allowlist (as reported in the second column of Table 2), all of them can be used to launch BAT from malicious centrals to track victim's centrals. When configuring the four development boards with RPAs, only TI CC2640 and Nordic NRF52 can be used to track both the central and the peripheral from a malicious central, since only they support the allowlist [32, 33].

Active BAT Attacks from Malicious Peripherals. Similarly, since the active BAT attacks from malicious peripherals leverage the *initiator filter policy*, which requires the central to add the peripherals to its allowlist for auto-connection, for each central we iterate our peripherals to pair it and then test whether the central will initiate the connection requests without manually triggering the connection in the Settings app. If so, the peripherals can be used to launch BAT attacks from malicious peripherals to track their centrals. As reported in Table 2, all of the tested devices are subject to tracking of peripherals (the last column), and tracking of centrals except the tested Wristband since its central does not use allowlist.

Table 4. Summary of Distribution of Sniffed BLE Devices

Item	Total		School Buildings		Student Union		Hospital		Residential		Plaza	
	#	%	#	%	#	%	#	%	#	%	#	%
All devices	100,101	100	15,804	15.7	19,254	19.2	5,648	5.6	48,802	48.7	10,593	10.5
Beacons	18,223	100	2,992	16.4	2,342	12.8	793	4.4	5,672	31.1	6,424	35.2
# Peripheral	89,795	100	13,454	14.9	18,563	20.7	5,424	5.7	42,925	47.8	9,429	10.5
# Central	9,719	100	2,216	22.8	648	6.7	201	2.0	5,558	57.2	1,096	11.2
Central & Peripheral	587	100	134	22.8	43	4.4	23	3.9	319	54.3	68	11.5
w/ PA	20,408	100	1,989	9.0	2,694	13.2	884	4.3	12,485	61.2	2,356	11.5
w/ SRA	5,473	100	814	14.8	1,727	31.6	369	6.7	2,349	42.9	214	3.9
w/ RPA	61,469	100	10,804	17.6	11,463	18.6	3,769	6.1	28,726	46.7	6,707	10.9
w/ NRPA	12,751	100	2,197	17.2	3,370	26.4	626	4.9	5,242	41.1	1,316	10.3
w/ WL	16,422	100	3,617	22.0	1,669	10.2	468	2.8	9,141	55.2	1,527	9.3
w/ WL & w/ the Central nearby	1,527	100	55	3.7	217	14.2	38	2.5	914	59.9	303	19.8

8.4 Field Test on BAT Attacks

With the sniffed packets, we are able to obtain many insights on the Bluetooth usage in practice, including the distributions of (i) Bluetooth device types (e.g., Beacons) and their roles (peripherals or centrals), (ii) MAC address types (e.g., PA and SRA), and (iii) the allowlisting peripherals, (iv) the identified allowlisted centrals and among them, (v) the allowlisted centrals across time. We present the details of these experimental results in the following.

Distribution of the Sniffed Bluetooth MAC Addresses. Among the captured 2,248,198 Bluetooth packets, as shown in Table 4, we identified 100,101 Bluetooth MAC addresses. Among them, 18,223 of them belong to Bluetooth beacons (identified by ADV_NONCONN_IND packet), 89,795 devices are peripherals only (identified by ADV_IND and ADV_DIRECT_IND packet), 9,719 are centrals only (identified by SCAN_REQ or CONNECT_REQ packet), and 587 are both peripherals and centrals (containing both ADV_IND and SCAN_REQ or CONNECT_REQ packets). We can also notice that among the five regions we sniffed, residential area has most of the devices. Also, unsurprisingly, the shopping area tends to have more beacons than the other ones.

Distribution of the MAC Address Types. Since we can determine the MAC address types (by checking Txadd and MSB), we also presented their distributions of the sniffed MAC addresses, which is reported between 6th row and 9th row in Table 4. We can see that there are 20,408 (20.3%) of all 100,101 MAC addresses with PA and 5,473 (5.4%) with SRA, which can be tracked without BAT. We identified 61,469 (61.4%) RPA addresses, meaning that the use of address randomization is indeed quite prevalent. The rest MAC addresses are NRPA, which accounted for 12.7% of all the MAC addresses.

Distribution of the allowlisting Peripherals. To identify whether a peripheral enables allowlist or not, we can just observe how SCAN_REQ is responded. In particular, we can first count the total number of SCAN_REQ (denoted by N_{req}) and SCAN_RSP (denoted by N_{rsp}) packets from a peripheral p , and then easily calculate its response ratio $\gamma = \frac{N_{rsp}}{N_{req}}$; if $\gamma < 1$, then we identify the desired allowlisting p , since the allowlisting peripheral will only respond to the known centrals.

As such, we need to first calculate γ for each identified peripheral. If $0 \leq \gamma < 1$, it is a allowlisting peripheral. With our calculated γ , we identified 16,422 whitelisting peripherals in total, which can be used to mount our BAT attacks. For these peripherals, we also identify that there are 1,527 centrals (which occupies 15.7% of all the identified 9,719 centrals) centrals are close to these allowlisting peripherals. Also, for these identified 16,422 allowlisting peripherals,

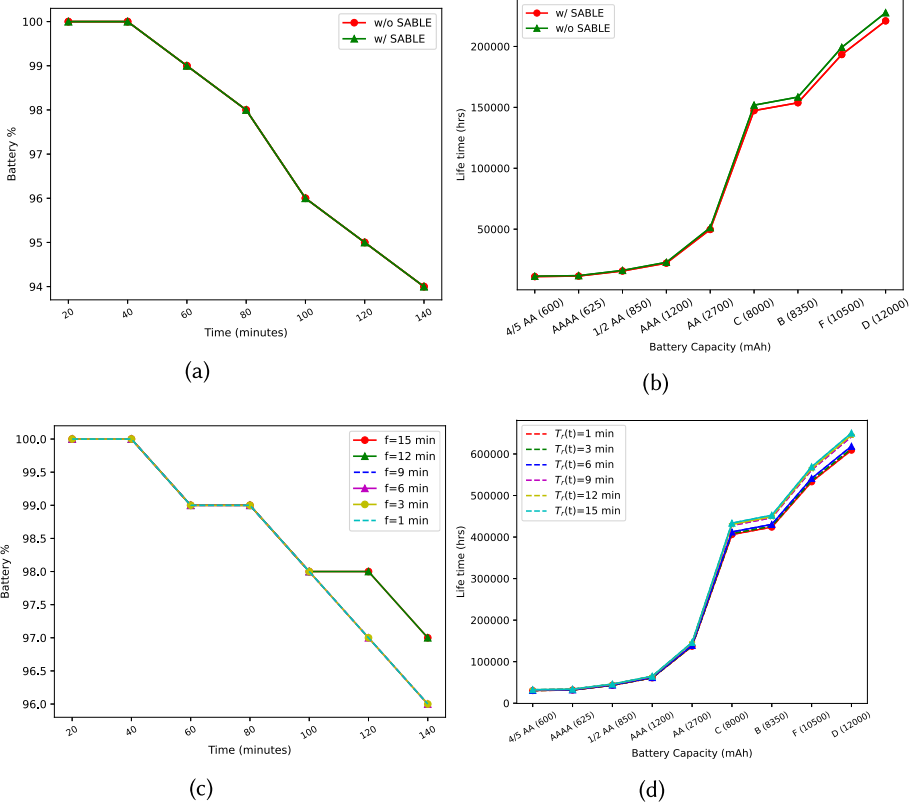


Fig. 8. Power consumption of SABLE defense.

we find that 3,909 of them (23.8%) used PAs and 644 of them (3.9%) used SRA, meaning that these devices usually do not change their MAC address, but 563 of them (3.4%) actually used NRPA (an untrackable address type), as shown in Table 3. However, we do observe most of these peripherals used RPA, which justifies the needs of our active attacks against these devices. Interestingly, among all the peripherals with allowlisting, 16,281 of them (99.1%) exclusively worked as the peripheral, and 141 of them worked as both central and peripheral. We also examined the type of the broadcasting packets from these allowlisting peripherals. As shown in the 8th row of Table 3, there are 797 peripherals that used ADV_DIRECT_IND advertising packets, which included the addresses of intended central [6]. There are 15,625 peripherals that only broadcasting ADV_IND.

Distribution of the Allowlisted Centrals. As shown in Table 4, our passive BAT attack eventually has identified 1,527 allowlisting peripherals with their whitelisted centrals nearby, and these devices can be de-anonymized. Among all the 1,527 allowlisted centrals (one peripheral is supposed to communicate with one central), we observed 228 allowlisted centrals mapped to 3 MAC

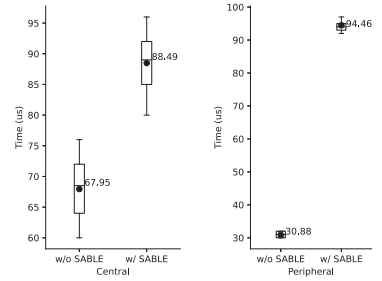


Fig. 9. Performance of SABLE for active defense

addresses, 310 allowlisted centrals mapped to 2 MAC addresses, and 141 mapped to 1 MAC address. Since we only sniffed the Bluetooth packets for 30 minutes, we did not observe more randomized MAC addresses mapping to one central.

(V) Distribution of Tracked allowlisted Centrals across Time. With the sniffed packets, we can also know the number of devices that can be tracked across our two visits, by using the identified PAs or SRAs of the peripherals to track the allowlisted centrals. We cannot use the RPAs since we do not know whether it is a new peripheral or not. There were 104 allowlisting peripherals that were observed in both the first time and the second time of our visit based on the observed PAs and SRAs. And among them, 98 peripherals used PA, and 6 devices used SRA. By analyzing their corresponding centrals, we linked 103 centrals across time (one peripheral did not have its central nearby).

8.5 Evaluation of SABLE

Effectiveness of SABLE. We launched both our passive and active attacks again against our patched Google Pixel 2 and NRF52, and confirmed that the attacks no longer work. In particular, (i) in passive attacks, where MAC addresses randomize on both sides in a synchronized random interval, attackers cannot observe predictable patterns between centrals and peripherals; (ii) in active attacks, SABLE rejected all reused packets, and attackers could not link any used RPAs to the new ones by replaying (including via tunnelling).

Power and Performance Overhead. Since our defense introduces extra operations, which will certainly result in both power consumption and performance overhead. To quantify the battery impact on central devices, we directly monitor it through the battery level. To quantify the battery impact on peripherals, we measured our tested NRF52 development board, with an Agilent 34410A Multi-meter to sample the electric current of NRF52 that runs with our SABLE (and without SABLE), and calculate the average current I_{avg} , from which to approximate the battery life, using the standard $T_{bat} = C_{bat}/I_{avg}$ [34], where C_{bat} is the battery capacity.

- **Active Defense.** Since our active defense piggybacks the existing pairing messages, we measured the approximation of the battery life of our defense and compared it with the vanilla pairing protocol. According to our experiment, the remaining battery percentage decreases linearly, and there is no extra overhead observed, as shown in Figure 8(a). Also, the average current of NRF52 running SABLE pairing process is $54.3 \mu A$, and the average current when NRF52 running vanilla pairing process is $52.7 \mu A$ (an extra 3.04% overhead). This overhead is negligible for a battery lifetime across different battery capacities, as illustrated in Figure 8(b). To measure the performance of our active defense, we run both our and vanilla RPA generation and resolution 100 times on both Google Pixel 2 and NRF52, respectively. Figure 9 shows these results. We can observe that the average overhead on NRF52 is around $63.58 (94.46-30.88) \mu s$ whereas the smartphone is around $20.54 (88.49-67.95) \mu s$.
- **Passive Defense.** For passive defense with synchronized random intervals, it does not have any additional overhead if its $T_r(t)$ is set to be a fixed 15 minutes (as in iOS) since it does not have any additional operations. However, if $T_r(t)$ becomes smaller, there will be more frequent address randomizations, causing more overhead. To quantify this power consumption, we set 15 minutes as the baseline, vary $T_r(t)$ from 3 to 15 with a stride 3. The overhead on the central device is around 1%, even if we set $T_r(t)=1$ (i.e., randomizing at every minute). For the peripheral, as shown in Figure 8(d), only 6.75% extra power consumption is introduced when $T_r(t)=1$, which is also negligible. For its performance, as shown in Figure 9, we can see that it takes around $88.49 \mu s$ and $94.46 \mu s$ for a mobile and development board to generate an address,

respectively (with our active defense). Therefore, compared to the address randomization interval (e.g., a few minutes), such a tiny amount of time is negligible.

9 Discussion

Attacks Beyond Address Tracking. Our BAT attacks can go beyond just the address tracking, and they can be combined with other attacks to further breach user's security and privacy. For instance, if the type of the allowlisting peripherals (e.g., a Blood glucose meter), which can be fingerprinted by techniques such as BLEScope [17], is known to the attackers, then he or she could further infer that the user of the associated centrals may be plagued by hyperglycemia or other blood glucose disorders. Also, if the devices use insecure pairing such as "Just Works", an attacker could even launch unauthorized data access to the targeted device.

Additionally, the attacker can also launch targeted attacks such as **Denial-of-Service (DoS)** attacks. One type of the DoS attacks is the battery draining of peripherals [35]. Specifically, allowlisting is supposed to secure the advertising stage of a BLE device and reduce its power consumption to only respond to the allowlisted device. However, an attacker can create a malicious central to keep connecting the vulnerable peripheral, thereby draining its battery. The other type of DoS attacks is to simply connect to the targeted peripherals. This is possible because many peripherals are only allowed to connect to one central at a time (e.g., a device that uses Bluetooth 4.0 or 4.1 [8, 11]). Once the malicious central connects the vulnerable peripheral, the peripheral will stop broadcasting and become invisible to other centrals. At this moment, the attacker can further deploy attacks such as unauthorized data access, if the peripheral uses Just Works pairing [36]. Likewise, these attacks can also be launched to against the vulnerable centrals from malicious peripherals. As shown in Table 6, most of our tested peripherals are subject to these attacks.

Practicality and Comparison. We have presented one passive BAT attack and two active BAT attacks. Next, we would like to compare the practicality (i.e., the cost, difficulty, and impact) of each attack. In particular, the cost is measured by the resources spent and also the efforts taken by the attackers; the difficulty is measured by the condition of the victim central and peripheral (e.g., whether the attacks require the two devices to be nearby); the impacts are the possible consequences caused by the attack. As summarized in Table 5, it can be observed that:

- **Passive Attacks (Section 4).** Since these attacks involve only sniffers, they are less costly. Meanwhile, the attacks do not require attackers to keep probing or building a tunnel. However, such attacks need uninterrupted monitoring of the traffic. Otherwise, the attacker will lose the target. Moreover, the attacks will fail when the central and peripheral are not within the reach.
- **Active Attacks from Malicious Centrals (Section 5.1).** When compared with the passive attacks, the active BAT attacks from a malicious central do not have the distance limit and do not require uninterrupted monitoring of the victim. Moreover, in addition to monitoring user behaviors, attacks can also be used to track a victim's past trajectory or instant location. However, such attacks are also subject to limitations: when launching the attacks, the attacker must probe multiple devices with SCAN_REQ or SCAN_RSP in order to identify the one of interest. Such operations can be costly when there are too many devices nearby.
- **Active Attacks from Malicious Peripherals (Section 5.2).** The active BAT attacks from malicious peripherals are most impactful, practical, and less costly. First, similar to the other active attacks, the attackers can launch such attacks without a distance limit and uninterrupted monitoring of the victims. Second, instead of probing each device independently, such attacks use ADV_IND, which is a broadcast message. That is, once the attacker broadcasts a single ADV_IND with an old MAC address, the victim central will expose itself instantly if it is nearby.

Table 5. Summary of Our Attacks

The BAT Attacks	Cost			Difficulty		Impacts
	Involved Malicious Devices	Uninterrupted Passively Monitoring	Keeping Actively Probing	Tunnelling	(Requiring Central and Peripheral to be Nearby)	
PA (Section 4)	Sniffer	✓	✗	✗	✓	①
AAC (Section 5.1)	Sniffer, Central, Peripheral	✗	✓	✓ & ✗	✗	①, ②, ③
AAP (Section 5.2)	Sniffer, Peripheral	✗	✗	✗	✗	①, ②, ③

PA means passive attacks. AAC means active attacks from malicious centrals. AAP means active attacks from malicious peripherals. ① means “monitoring the user’s behavior”, ② means “tracking a victim’s past trajectory”, and ③ means “tracking a victim’s real time location”.

Security Implications. Our work carries significant security implications. While not all Bluetooth peripherals support allowlists (typically, keyboards, mice, and smart speakers do), our study reveals that nearly every smartphone, whether running on Google’s Android or Apple’s iOS, enables this feature. Given the widespread use of smartphones worldwide (there are over 6.5 billion smartphones worldwide [37]), a substantial user base is susceptible to our attack. However, rectifying this issue is not a straightforward endeavor. Smartphone vendors like Google and Apple can swiftly release patches with countermeasures, but the peripheral side often lacks online updating mechanisms. While online updates can potentially safeguard the system, the effectiveness of these patches depends on the timely deployment of updates and their coverage across the device ecosystem. Some peripherals, such as certain smartwatches or wireless headphones, have adopted online updating systems that allow manufacturers to address vulnerabilities post-production. However, the challenge arises when such mechanisms are either absent or inconsistently applied. Furthermore, the situation becomes more complex with certain IoT devices that have limited computational resources and power consumption constraints. These devices, designed for efficiency and longevity, often lack the capability to handle frequent or large-scale security updates. The minimal processing power in these IoT peripherals restricts their ability to run complex cryptographic algorithms or adaptive security measures, leaving them particularly vulnerable to attacks.

Another aspect to consider is the presence of an active defense mechanism that is aware of such attacks. Active defense systems, often integrated into smartphones, networks, or even certain IoT ecosystems, could play a pivotal role in mitigating these attacks. If such a defense mechanism possesses real-time knowledge of the attack vectors we highlight, it could employ techniques like anomaly detection, behavioral analysis, or active monitoring to preemptively block or disrupt suspicious activity [38]. For example, modern **intrusion detection systems (IDS)** could flag unusual Bluetooth behavior, such as unauthorized pairings or abnormal packet transmissions, to quarantine the threat. However, active defense systems are not without their own challenges. First, it is important to emphasize that our passive attack does not involve the transmission of any packets. Instead, it relies solely on passively sniffing communications between devices. From the perspective of active defense mechanisms, this presents a unique challenge. Since our attack does not generate any abnormal traffic, such as unauthorized packet transmissions, unexpected device pairing attempts, or modifications to the communication stream, it remains undetectable by conventional IDS or anomaly-based monitoring tools. Second, in the context of our active defense, the attack only requires the transmission of a single packet, such as a SCAN_REQ, to which the victim device responds with a SCAN_RSP. From this perspective, the attacker’s behavior is indistinguishable from that of a legitimate central device. While a legitimate central device may subsequently initiate a connection request to the victim device, leading to further communication between the two, this additional step is not mandatory for the attack. In fact, many legitimate devices in close proximity can simply exchange SCAN_REQ and SCAN_RSP packets without proceeding to a full connection. Since the exchange of SCAN_REQ and SCAN_RSP packets is part of normal BLE discovery processes,

it becomes challenging for active defense mechanisms to differentiate between an attacker and a legitimate central device based solely on this interaction.

Security Analysis of Our Defenses. In our passive BAT defense, we rely on synchronized random time intervals at both central and peripheral to defeat cross interval MAC address tracking. Intuitively, since the defense requires synchronized random time intervals to work, and if the attacker can manipulate intervals by removing (e.g., through jamming) or injecting packets that are required for both the central and peripheral to execute the synchronization, it will disable our defense. However, while it is true that the manipulation may result in the DoS attack, the attacker cannot further track the central or the peripheral. Since the central and the peripheral can be far away from or close to each other (See Section 6.1), we then have the following two cases to analyze:

- **Two devices are far away.** In this case, the two devices independently perform their address randomization, and SCAN_REQ or SCAN_RSP sent from the two devices will not be used to perform the synchronization. As such, if the attacker removes some or all of those packets, the devices will behave as normal, no impact on the random time intervals. Meanwhile, since the attacker does not have the corresponding IRK, he or she cannot create and forge a packet with a valid MAC address. The attacker may also replay an used packet, but it is prevented by our active defense.
- **Two devices are close to each other.** In this case, the devices perform randomization according to the exchanged SCAN_REQ and SCAN_RSP. First, removing some of those packets intentionally may result in the failure of the synchronization (e.g., it is supposed to be the i th interval, but the peripheral or the central may still be in the previous ($i-1$)-th interval). However, an error correction mechanism can make the random time intervals to be re-synchronized. Second, forging or replaying the packets will not work too, the reasons of which are similar to the previous case, thereby omitted for brevity.

In an extreme case, the attacker may first remove some of the packets from the traffic, and then replay those “removed” packets, in the purpose of breaking our BAT active defense. Such attack will not work as well. Assume that at t_0 , the attacker first removes all the packets sent from the peripheral, and then saves those packets. At t_1 (i.e., $t_0 + T$), the attacker sends out the collected packets to probe the central(s). The “true” central will be able

Table 6. Targeted Attacks Against Allowlisting Peripherals

Device Name	Pairing Method	Denial of Service	Unauthorized Data Access
Jelly Comb Keyboard	Just Works	✓	✓
iClever Keyboard	Just Works	✓	✓
Microsoft Designer Keyboard	Passkey Entry	✗	✗
Microsoft Clavier Keyboard	Passkey Entry	✗	✗
Byteblue Keyboard	Just Works	✓	✓
Logitech K780 Keyboard	Passkey Entry	✗	✗
Logitech K830 Keyboard	Passkey Entry	✗	✗
DRACONIC Keyboard	Just Works	✓	✓
SXWL Keyboard	Just Works	✓	✓
SXWL Mouse	Just Works	✓	✓
Inphic Mouse	Just Works	✓	✓
VOGEK Mouse	Just Works	✓	✓
Jelly Comb V1 Mouse	Just Works	✓	✓
Jelly Comb V2 Mouse	Just Works	✓	✓
SEENDA Mouse	Just Works	✓	✓

to resolve it, and quickly notice those packets are old ones due to the significant differences between those packets and the current time (though those packets have never been observed before), thereby still defeating such attacks. Certainly, if the attacker obtains the IRK of the central or the peripheral, our defense will fail since he or she can forge the MAC address based on our proposed randomization algorithm. However, we consider this to be hard since the transmission of the IRK is protected by encryption.

Mitigating BAT on Devices with Static Addresses. Some peripherals may use static MAC addresses (e.g., PA or an SRA). Then it allows attackers to easily associate the peer's MAC addresses when observing it responds to the listed and non-listed devices differently once configured with allowlist. Therefore, to mitigate this side channel, we may adopt a semi-allowlist scheme to also randomly (controlled by a parameter λ) respond to the non-listed devices but not all of them (otherwise it becomes non-allowlist).

The semi-allowlist scheme is quite simple. It basically disables the attackers to observe the patterns of always responding to allowlisted devices but ignoring the non-listed devices. At a high level, it uses a parameter λ to decide how often a peripheral p should respond to the non-listed devices. Since using allowlist has the consideration of battery-saving, we should not let p respond to all the packets ($\lambda = 1$) and instead we can let it respond for example half of them ($\lambda = 1/2$) or even less. Also, if p has responded to a central c during c 's randomization interval, it should keep responding c until the time of this interval is over; otherwise attackers can easily identify c is not in its allowlist. On the other hand, if $\lambda = 0$, then it becomes allowlist and attackers can link the randomized allowlisted centrals across time. Additionally, more centrals nearby, less responses from the peripheral. Therefore, λ is a dynamic value depending on the environment, and our evaluation will characterize the impact of this parameter.

However, this can enable a wide range of privacy attacks that will be even more catastrophic, since this allows the Bluetooth devices to connect other devices without users initiating the connections. As such, we still recommend devices to use randomization on both sides, which has even less footprint. Suppose a device can resolve the randomized addresses from its peers. In that case, it must have the capabilities to generate a new address for its own since the algorithms to resolve and generate randomized MAC addresses are the same. Before our attacks, some devices such as keyboards do not use address randomization since the keyboards always stay at the exact location, which usually does not concern tracking. However, in our article, we show that such devices can be used to mount tracking attacks, through which the attackers can potentially monitor the victim's status or track a victim's trajectory. Therefore, enforcing the randomization at both sides is a more practical way.

Ethics and Responsible Disclosure. We took the highest ethical standards when conducting our BAT attacks, with several key measures in place to mitigate concerns about capturing users' confidential data. First, we only passively listened to over-the-air Bluetooth packets, ensuring that no packets were modified or injected in ways that could harm users or interfere with their devices. This approach greatly minimized the risk of tampering with sensitive information. Second, we strictly limited our analysis to advertisement packets, which contain publicly broadcasted, non-sensitive data intended for device discovery. Any other types of packets, especially those that could contain private or sensitive information, were immediately deleted to prevent any inadvertent exposure of users' confidential data. Additionally, packets beyond these, such as those used for actual communication between devices, are typically encrypted, further reducing the risk of exposing sensitive data. Any non-broadcasting packets that were unintentionally captured were promptly deleted to ensure no confidential information was exposed or analyzed. Third, we ensured that active attacks were only performed on our own devices, completely avoiding the risk of impacting external users or compromising their privacy. Moreover, we followed responsible disclosure protocols and reported the vulnerability to key industry players, including Bluetooth SIG, Apple, Google, Microsoft, Texas Instruments, and Nordic Semiconductor. Bluetooth SIG assigned CVE-2020-35473 to track this logical-level design flaw, and Google assigned Android-ID 175212130, rating our vulnerability as the second-highest severity among its five-level system. Google also rewarded us with a bug bounty. Apple assigned internal vulnerability-ID 755406462, and Microsoft

assigned case tracking-ID MSRC-63104. TI informed us that they were following Bluetooth SIG's recommendations for addressing the vulnerability.

10 Related Work

Bluetooth Privacy. Over the past 20 years, numerous Bluetooth device tracking attacks have been proposed by collecting the advertising packets (e.g., ADV_IND, SCAN_REQ) using sniffers [1–5, 12, 13, 23–25, 39–41], as summarized in Table 7. Among them, earlier efforts such as BlueTrack [12] and BLEB [13] track the Bluetooth devices that use PAs. Marco et al. [25] track the Bluetooth classic devices that do not use address randomization via the information leaked from the frame encoding. However, when the devices use randomized MAC addresses, although intuitively one can associate a disappeared address and a newly appeared address to break the address randomization, one location can have multiple devices, which can appear or disappear simultaneously (e.g., turning on/off devices, devices moving out/in the range of sniffers). As such, monitoring the devices solely will have high false alerts, and extra efforts are needed for the tracking. To this end, Ludant et al. [24] linked and tracked the BLE addresses via the collection of Bluetooth Classic (BT) Addresses. Our attacks instead do not need to collect both Bluetooth classic packets (which are hard to be collected as discussed in [25]) and BLE packets, and we only need to collect BLE advertising packets.

There are also tracking attacks against specific implementations of Bluetooth devices (e.g., Apple devices [40–43] and wearable fitness trackers [45]). Some attacks exploit static payloads (e.g., fixed manufacture identifiers [2, 3], fixed information elements [46, 47], and fixed GATT attribute [48])) to track devices. Most of those attacks are implementation specific (which usually requires unchanged payload), whereas our BAT attacks are based on protocol level flaws we identified. There are also tracking attacks requiring the attacker to obtain keys (e.g., [23, 44, 49]). Our attacks do not make such assumption. Moreover, unlike the traditional tracking attacks [3, 16], where the attacker has to collect many BLE packets and analyze them one by one to identify the victim, our attack from malicious peripherals, for example, replays an old packet and the victim will be exposed instantly.

Bluetooth Security. Bluetooth devices are also subject to multiple security attacks such as brute force attacks against PIN (e.g., [50, 51]), eavesdropping attacks (e.g., [52–54]), MiTM attacks [36, 55–57], and vulnerable pairing (e.g., [21, 58, 59]). In addition to these link layer attacks, there are also efforts targeted on Bluetooth applications. For example, the Bluetooth Misbonding Attacks [19] target peripherals that do not enforce application layer authentication and

Table 7. Comparison of Our BAT Attacks with Others

Attacks	Non-Implementation Specific					
	Against RPA	w/o Unchanged Payloads	w/o Installing Malware	w/ either BLE or BT Packets	w/o Stealing IRK/Key	
Jakobsson et al. [1]	✗	✗	✗	✓	✓	✓
BlueTrack [12]	✗	✗	✗	✓	✓	✓
BLEB [12]	✗	✗	✗	✓	✓	✓
BLE-Guardian [2]	✗	✓	✗	✓	✓	✓
Korolova et al. [5]	✗	✓	✗	✗	✓	✓
Becker et al. [3]	✗	✓	✗	✓	✓	✓
Martin et al. [41]	✗	✓	✗	✓	✓	✓
Celosia et al. [4]	✗	✓	✗	✓	✓	✓
BLEScope [17]	✗	✗	✗	✓	✓	✓
Stute et al.(2019) [42]	✗	✓	✗	✓	✓	✓
Marco et al. [25]	✗	✗	✓	✓	✓	✓
Zhang et al. [23]	✗	✓	✓	✓	✓	✗
FirmwXray [39]	✗	✗	✓	✓	✓	✓
Guillaume et al. [40]	✗	✓	✗	✓	✓	✓
Cominelli et al. [25]	✗	✗	✗	✓	✓	✓
Stute et al.(2021) [43]	✗	✓	✗	✓	✓	✓
Heinrich et al. [44]	✗	✓	✓	✓	✓	✗
Ludant et al. [24]	✗	✓	✗	✓	✗	✓
BAT Attacks	✓	✓	✓	✓	✓	✓

BadBluetooth [18] exploits the weakness when the central does not authenticate the peripherals. Compared to these works, we are the first to exploit the allowlist in Bluetooth protocol for device tracking.

Prior attacks in wireless sensor networks could also be applied to Bluetooth. For instance, in our Attack Example III, we have leveraged the concept of the wormhole attacks [27] to establish a tunnel between a sniffer and the malicious central located nearby of the stationary peripheral. While wormhole attack alone could be used to tunnel connections between a central and a peripheral to identify a device, it would require additional information such as our allowlisting side channels when there are multiple devices nearby.

IoT Devices Security. Numerous studies have also focused on IoT security [60–63], particularly on tracking or fingerprinting IoT devices [64–67]. For instance, the leaked information (e.g., the network connection frequency, the DNS server the device connects) in the encrypted traffic can be used to fingerprint IoT devices, or directly analyzing network traces with machine learning algorithms to identify IoT devices (e.g., [65–67]). Recently, Huang et al. [68] even showed that an attacker may observe changes in IoT device state and infer the users' behaviors and lifestyles, based on just IoT network traffic. Compared to these works, we track Bluetooth devices based on the distinctive traffic behavior caused by the trust relationship (i.e., allowlisting).

11 Conclusion

We have presented BAT attacks by exploiting the single-bit of side channel of response and no-response from BLE devices with allowlist to track their randomized MAC addresses. Our attacks can be launched passively, or actively due to the vulnerability of current RPA generation and resolution algorithm. We have also presented defense SABLE, by adding a random sequence number when generating and resolving the RPA-type MAC addresses to defeat our active BAT attacks, and enforcing an interval unpredictable, central and peripheral synchronized RPA generation scheme to counter passive attacks. The experimental results show that our defense has negligible impact on both the battery consumption and performance overhead for the involved centrals and peripherals.

Acknowledgments

We would like to thank Shuo Chen, Syed Rafiul Hussain, Moti Yung, and the anonymous reviewers for their helpful comments on an early draft of this article.

References

- [1] Markus Jakobsson and Susanne Wetzel. 2001. Security weaknesses in Bluetooth. In *Proceedings of the Cryptographers' Track at the RSA Conference*. Springer, 176–191.
- [2] Kassem Fawaz, Kyu-Han Kim, and Kang G. Shin. 2016. Protecting privacy of BLE device users. In *Proceedings of the 25th USENIX Security Symposium (USENIX Security 16)*. 1205–1221.
- [3] Johannes K. Becker, David Li, and David Starobinski. 2019. Tracking anonymized Bluetooth devices. *Proceedings on Privacy Enhancing Technologies (PETS)* 2019, 3 (2019), 50–65.
- [4] Guillaume Celosia and Mathieu Cunche. 2019. Saving private addresses: An analysis of privacy issues in the Bluetooth-low-energy advertising mechanism. In *Proceedings of the 16th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*. 444–453.
- [5] Aleksandra Korolova and Vinod Sharma. 2018. Cross-app tracking via nearby Bluetooth low energy devices. In *Proceedings of the 8th ACM Conference on Data and Application Security and Privacy (CODASPY)*. 43–52.
- [6] SIG Bluetooth. 2014. Bluetooth core specification version 4.2. *Specification of the Bluetooth System* (2014).
- [7] Apple Inc. 2019. Accessory Design Guidelines for Apple Devices). Retrieved July 5, 2025 from <https://developer.apple.com/accessories/Accessory-Design-Guidelines.pdf>. (2019).
- [8] SIG Bluetooth. 2024. *Bluetooth Core Specification Version 6.0: Specification of the Bluetooth System* (2024).
- [9] SIG Bluetooth. 2024. *Bluetooth Core Specification Version 6.0: Specification of the Bluetooth System* (2024).

- [10] Google. 2025. Android Open Source Project. Retrieved from <https://source.android.com/>. ([n. d.]).
- [11] SIG Bluetooth. 2024. *Bluetooth Core Specification Version 6.0: Specification of the Bluetooth System* (2024).
- [12] Marc Haase, Matthias Handy, et al. 2004. BlueTrack—imperceptible tracking of Bluetooth devices. In *UbiComp Poster Proceedings*, Vol. 2.
- [13] Taher Issoufaly and Pierre Ugo Tournoux. 2017. BLEB: Bluetooth low energy botnet for large scale individual tracking. In *Proceedings of the 2017 1st International Conference on Next Generation Computing Applications (NextComp)*. IEEE, 115–120.
- [14] Google. 2016. Android 6.0 Changes. Retrieved July 5, 2025 from <https://developer.android.com/about/versions/marshmallow/android-6.0-changes#behavior-hardware-id>. (2016).
- [15] Christopher Ellis, Haohuang Wen, Zhiqiang Lin, and Anish Arora. 2022. Replay (far) away: Exploiting and fixing Google/Apple exposure notification contact tracing. *Proceedings on Privacy Enhancing Technologies (PETS) 2022*, 4 (2022), 727–745.
- [16] Célestin Matte, Mathieu Cunche, Franck Rousseau, and Mathy Vanhoef. 2016. Defeating MAC address randomization through timing attacks. In *Proceedings of the 9th ACM Conference on Security & Privacy in Wireless and Mobile Networks*. 15–20.
- [17] Chaoshun Zuo, Haohuang Wen, Zhiqiang Lin, and Yinqian Zhang. 2019. Automatic fingerprinting of vulnerable BLE IoT devices with static UUIDs from mobile apps. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 1469–1483.
- [18] Fenghao Xu, Wenrui Diao, Zhou Li, Jiongyi Chen, and Kehuan Zhang. 2019. BadBluetooth: Breaking android security mechanisms via malicious Bluetooth peripherals. In *Proceedings of the 26th Annual Network and Distributed System Security Symposium (NDSS'19)*.
- [19] Muhammad Naveed, Xiao-yong Zhou, Soteris Demetriou, XiaoFeng Wang, and Carl A. Gunter. 2014. Inside job: Understanding and mitigating the threat of external device mis-binding on android. In *Proceedings of the 21st Annual Network and Distributed System Security Symposium, NDSS 2014*.
- [20] Lily Hay Newman. 2020. Sneaky Zero-Click Attacks Are a Hidden Menace. Retrieved December 24, 2021 from <https://www.wired.com/story/sneaky-zero-click-attacks-hidden-menace/>. (April 2020).
- [21] Daniele Antonioli, Nils Ole Tippenhauer, and Kasper Rasmussen. 2020. BIAS: Bluetooth impersonation attacks. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*.
- [22] Daniele Antonioli, Nils Ole Tippenhauer, and Kasper B. Rasmussen. 2019. The {KNOB} is broken: Exploiting low entropy in the encryption key negotiation of Bluetooth BR/EDR. In *Proceedings of the 28th {USENIX} Security Symposium ({USENIX} Security 19)*. 1047–1061.
- [23] Yue Zhang, Jian Weng, Rajib Dey, Yier Jin, Zhiqiang Lin, and Xinwen Fu. 2020. Breaking secure pairing of Bluetooth low energy using downgrade attacks. In *Proceedings of the 29th {USENIX} Security Symposium ({USENIX} Security 20)*. 37–54.
- [24] Norbert Ludant, Tien D. Vo-Huu, Sashank Narain, and Guevara Noubir. 2021. Linking Bluetooth LE & classic and implications for privacy-preserving Bluetooth-based protocols. In *Proceedings of the 2021 IEEE Symposium on Security and Privacy (SP)*.
- [25] Marco Cominelli, Francesco Gringoli, Paul Patras, Margus Lind, and Guevara Noubir. 2020. Even black cats cannot stay hidden in the dark: Full-band de-anonymization of Bluetooth classic devices. In *Proceedings of the 2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 534–548.
- [26] SENA. 2025. Patch Antenna - RP-SMA-R/A Right-Hand Thread, 9dBi. 636–645. Retrieved February 2025 from <http://www.senanetworks.com/pat-g01r.html>. (2025).
- [27] Yih-Chun Hu, Adrian Perrig, and David B. Johnson. 2006. Wormhole attacks in wireless networks. *IEEE Journal on Selected Areas in Communications* 24, 2 (2006), 370–380.
- [28] Leslie Lamport. 2019. Time, clocks, and the ordering of events in a distributed system. In *Concurrency: The Works of Leslie Lamport*. Association for Computing Machinery (ACM). 179–196.
- [29] Statistics. 2020. Bluetooth device shipments worldwide from 2015 to 2026. Retrieved November 19, 2014 from <https://www.statista.com/statistics/1220933/global-bluetooth-device-shipment-forecast/>. (April 2020).
- [30] Adafruit . Adafruit Sniffer. Retrieved from <https://learn.adafruit.com/introducing-the-adafruit-bluefruit-le-sniffer/introduction>. ([n. d.]).
- [31] Maxim Chernyshev, Craig Valli, and Michael Johnstone. 2017. Revisiting urban war nibbling: Mobile passive discovery of classic Bluetooth devices using Ubetooth one. *IEEE Transactions on Information Forensics and Security* 12, 7 (2017), 1625–1636.
- [32] Ganesh. 2020. The current PSoc EZ-Serial FW does not support RPA for whitelist. Retrieved July 5, 2025 from <https://community.cypress.com/thread/51728?start=0&tstart=0>. (2020).
- [33] Silcon Labs. 2020. TWhitelisting (Silcon Labs Official Document). Retrieved July 5, 2025 from <https://docs.silabs.com/bluetooth/3.0/general/adv-and-scanning/whitelisting>. (2020).

- [34] Eduardo Garcia-Espinosa, Omar Longoria-Gandara, Ioseth Pegueros-Lepe, and Arturo Veloz-Guerrero. 2018. Power consumption analysis of Bluetooth low energy commercial products and their implications for IoT applications. *Electronics* 7, 12 (2018), 386.
- [35] Jason Uher, Ryan G. Mennecke, and Bassam S. Farroha. 2016. Denial of sleep attacks in Bluetooth low energy wireless sensor networks. In *MILCOM 2016-Proceedings of the 2016 IEEE Military Communications Conference*. IEEE, 1231–1236.
- [36] Konstantin Hypponen and Keijo M. J. Haataja. 2007. “Nino” man-in-the-middle attack on Bluetooth secure simple pairing. In *Proceedings of the 2007 3rd IEEE/IFIP International Conference in Central Asia on Internet*. IEEE, 1–5.
- [37] Josh Howarth. 2025. How Many People Own Smartphones? (2024-2029). Retrieved June 18, 2025 from <https://explodingtopics.com/blog/smartphone-stats>. (2025).
- [38] Xiao Li, Yue Li, Hao Wu, Yue Zhang, Kaidi Xu, Xiuzhen Cheng, Sheng Zhong, and Fengyuan Xu. 2025. Make a feint to the east while attacking in the west: Blinding LLM-based code auditors with flashboom attacks. In *Proceedings of the 2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 539–557.
- [39] Hanghuang Wen, Zhiqiang Lin, and Yinqian Zhang. 2020. FirmXRay: Detecting Bluetooth link layer vulnerabilities from bare-metal firmware. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*.
- [40] Guillaume Celosia and Mathieu Cunche. 2020. Discontinued privacy: Personal data leaks in Apple Bluetooth-low-energy continuity protocols. *Proceedings on Privacy Enhancing Technologies* 2020, 1 (2020), 26–46.
- [41] Jeremy Martin, Douglas Alpuche, Kristina Bodeman, Lamont Brown, Ellis Fenske, Lucas Foppe, Travis Mayberry, Erik Rye, Brandon Sipes, and Sam Teplov. 2019. Handoff all your privacy—A review of Apple’s Bluetooth low energy continuity protocol. *Proceedings on Privacy Enhancing Technologies* 2019, 4 (2019), 34–53.
- [42] Milan Stute, Sashank Narain, Alex Mariotto, Alexander Heinrich, David Kreitschmann, Guevara Noubir, and Matthias Hollick. 2019. A billion open interfaces for eve and mallory: {MitM}, {DoS}, and tracking attacks on {iOS} and {macOS} through Apple wireless direct link. In *Proceedings of the 28th USENIX Security Symposium (USENIX Security 19)*, 37–54.
- [43] Milan Stute, Alexander Heinrich, Jannik Lorenz, and Matthias Hollick. 2021. Disrupting continuity of Apple’s wireless ecosystem security: New tracking, {DoS}, and {MitM} attacks on {iOS} and {macOS} through Bluetooth low energy, {AWDL}, and {Wi-Fi}. In *Proceedings of the 30th USENIX Security Symposium (USENIX Security 21)*. 3917–3934.
- [44] Alexander Heinrich, Milan Stute, Tim Kornhuber, and Matthias Hollick. 2021. Who can devices? Security and privacy of Apple’s crowd-sourced Bluetooth location tracking system. *Proceedings on Privacy Enhancing Technologies* 2021, 3 (2021), 227–245.
- [45] Aveek K. Das, Parth H. Pathak, Chen-Nee Chuah, and Prasant Mohapatra. 2016. Uncovering privacy leakage in BLE network traffic of wearable fitness trackers. In *Proceedings of the 17th International Workshop on Mobile Computing Systems and Applications*. ACM, 99–104.
- [46] Mathy Vanhoef, Célestin Matte, Mathieu Cunche, Leonardo S. Cardoso, and Frank Piessens. 2016. Why MAC address randomization is not enough: An analysis of Wi-Fi network discovery mechanisms. In *Proceedings of the 11th ACM on Asia Conference on Computer and Communications Security*. 413–424.
- [47] Haohuang Wen, Qingchuan Zhao, Zhiqiang Lin, Dong Xuan, and Ness Shroff. 2020. A study of the privacy of COVID-19 contact tracing apps. In *Proceedings of the International Conference on Security and Privacy in Communication Networks*.
- [48] Guillaume Celosia and Mathieu Cunche. 2019. Fingerprinting Bluetooth-low-energy devices based on the generic attribute profile. In *Proceedings of the 2nd International ACM Workshop on Security and Privacy for the Internet-of-Things*. 24–31.
- [49] Alexander Heinrich, Milan Stute, and Matthias Hollick. 2021. OpenHaystack: A framework for tracking personal Bluetooth devices via Apple’s massive find my network. In *Proceedings of the 14th ACM Conference on Security and Privacy in Wireless and Mobile Networks*. 374–376.
- [50] Andreas Becker and Ing Christof Paar. 2007. Bluetooth Security & Hacks. *Ruhr-Universität Bochum*.
- [51] Yaniv Shaked and Avishai Wool. 2005. Cracking the Bluetooth pin. In *Proceedings of the 3rd International Conference on Mobile Systems, Applications, and Services*. ACM, 39–50.
- [52] Dominic Spill and Andrea Bittau. 2007. BlueSniff: Eve meets alice and Bluetooth. *WOOT 7* (2007), 1–10.
- [53] Mike Ryan. 2013. Bluetooth: With low energy comes low security. In *Proceedings of the 7th USENIX Conference on Offensive Technologies (WOOT’13)*. USENIX Association, Berkeley, CA, USA, 4–4. Retrieved from <http://dl.acm.org/citation.cfm?id=2534748.2534754>
- [54] SIG Bluetooth. 2024. *Bluetooth Core Specification Version 6.0: Specification of the Bluetooth System* (2024).
- [55] Dennis Kügler. 2003. “Man in the Middle” attacks on Bluetooth. In *Proceedings of the International Conference on Financial Cryptography*. Springer, 149–161.
- [56] Keijo Haataja and Pekka Toivanen. 2010. Two practical man-in-the-middle attacks on Bluetooth secure simple pairing and countermeasures. *IEEE Transactions on Wireless Communications* 9, 1 (2010).

- [57] Yue Zhang, Jian Weng, Zhen Ling, Bryan Pearson, and Xinwen Fu. 2020. BLESS: A BLE application security scanning framework. In *IEEE INFOCOM 2020-Proceedings of the IEEE Conference on Computer Communications*. IEEE, 636–645.
- [58] SIG Bluetooth. 2024. *Bluetooth Core Specification Version 6.0: Specification of the Bluetooth System* (2024).
- [59] Daniele Antonioli, Nils Ole Tippenhauer, and Kasper Rasmussen. 2020. Key negotiation downgrade attacks on Bluetooth and Bluetooth low energy. *ACM Transactions on Privacy and Security (TOPS)* 23, 3 (2020), 1–28.
- [60] Chongqing Lei, Zhen Ling, Yue Zhang, Yan Yang, Junzhou Luo, and Xinwen Fu. 2024. A friend's eye is a good mirror: Synthesizing {MCU} peripheral models from peripheral drivers. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*. 7085–7102.
- [61] Kaizheng Liu, Ming Yang, Zhen Ling, Yue Zhang, Chongqing Lei, Junzhou Luo, and Xinwen Fu. 2024. RiOTFuzzer: Companion app assisted remote fuzzing for detecting vulnerabilities in IoT devices. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2341–2354.
- [62] Yue Zhang, Zhen Ling, Michael Cash, Qiguang Zhang, Christopher Morales-Gonzalez, Qun Zhou Sun, and Xinwen Fu. 2024. Collapse like a house of cards: Hacking building automation system through fuzzing. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 1761–1775.
- [63] Ye Cheng, Minghui Xu, Yue Zhang, Kun Li, Ruoxi Wang, and Lian Yang. 2025. AutoIoT: Automated IoT platform using large language models. *IEEE Internet of Things Journal* 12, 10 (2025).
- [64] Christopher Ellis, Yue Zhang, Mohit Kumar Jangid, Shixuan Zhao, and Zhiqiang Lin. 2025. Deanonymizing device identities via side-channel attacks in exclusive-use IoTs & mitigation. In *2025 Proceedings of the Annual Network and Distributed System Security Symposium*.
- [65] Yair Meidan, Michael Bohadana, Asaf Shabtai, Juan David Guarnizo, Martín Ochoa, Nils Ole Tippenhauer, and Yuval Elovici. 2017. ProfilloT: A machine learning approach for IoT device identification based on network traffic analysis. In *Proceedings of the Symposium on Applied Computing*. 506–509.
- [66] Roberto Perdisci, Thomas Papastergiou, Omar Alrawi, and Manos Antonakakis. 2020. IoTFinder: Efficient large-scale identification of IoT devices via passive DNS traffic analysis. In *Proceedings of the 2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 474–489.
- [67] Lingjing Yu, Bo Luo, Jun Ma, Zhaoyu Zhou, and Qingyun Liu. 2020. You are what you broadcast: Identification of mobile and IoT devices from (public) wifi. In *Proceedings of the 29th {USENIX} Security Symposium ({USENIX} Security 20)*. 55–72.
- [68] Danny Yuxing Huang, Noah Aporthe, Frank Li, Gunes Acar, and Nick Feamster. 2020. IoT inspector: Crowdsourcing labeled network traffic from smart home devices at scale. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 4, 2 (2020), 1–21.

Received 12 December 2022; revised 4 October 2024; accepted 21 May 2025